# Monkey See, Can Monkey Do? A Benchmark for Evaluating Robot Skill Learning by Observation

**Weiwei Gu***
School of Augmented Intelligence
Arizona State University
weiweigu@asu.edu

**Anmol Gupta***
School of Augmented Intelligence
Arizona State University
agupt374@asu.edu

**Anant Sah***
School of Augmented Intelligence
Arizona State University
asah4@asu.edu

**Ryan Varghese**
School of Augmented Intelligence
Arizona State University
rsvargh2@asu.edu

**Lalitha Shreya Vanam**
School of Augmented Intelligence
Arizona State University
lvanam@asu.edu

**Prabhath Adireddi**
School of Augmented Intelligence
Arizona State University
padiredd@asu.edu

**Peter Karkus**
NVIDIA
karkuspeter@gmail.com

**Nakul Gopalan**
School of Augmented Intelligence
Arizona State University
nakul.gopalan@asu.edu

**Abstract:** Learning from Observation (LfO) is a fundamental robotic capability that replicates how humans and animals socially learn from each other. Beyond its biological parallels, this modality provides a practical solution for data scaling in sample-inefficient and data-starved domains like robotics. Recent work has demonstrated promising results in learning manipulation skills from human videos, yet progress in this area remains difficult to assess. Existing methods vary widely in assumptions, hardware choices, and environment setups making it difficult to draw meaningful comparisons and identify advances in the field. To address these challenges, we introduce RoboReel: a unified benchmark for evaluating models that learn policies from human videos. RoboReel consists of bundled real-world human demonstration videos, simulated robot trajectories, and evaluation environments on ten manipulation tasks. We develop four test suites to evaluate the models' performance on multiple axes, including the robustness to visual distractors and the ability to complete long-horizon tasks. Our benchmark covers learning-from-observation models from different categories, and studies the effectiveness of multiple representation choices in our benchmark evaluation that covers over seven state-of-the-art algorithms (including our VLA based variants) in the field of LfO. Finally, we present an analysis of the different types of algorithms showing that long-horizon tasks and tasks with low tolerances are still challenging for current models. Webpage: https://roboreel.github.io

## 1 Introduction

Observational Learning is an important developmental skill for humans [1, 2, 3], other mammals [3, 4] and even birds [5]. It is no surprise that this learning paradigm, specifically from human

*Equal contribution; authors are listed in alphabetical order.

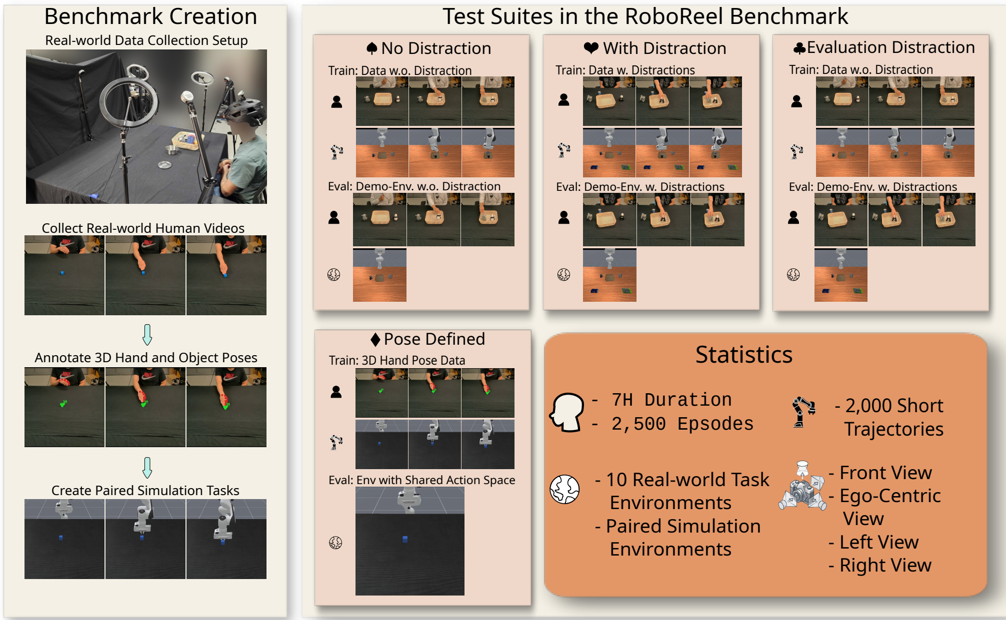


Figure 1: Overview – RoboReel consists of real-world data collected by six participants using a calibrated, tabletop setup across ten distinct tasks with over seven hours of data. To facilitate equitable lateral comparisons of multiple Learning from Observation (LfO) algorithms, we constructed a paired digital-twin simulation environment featuring identical objects with matching textures and colors. The benchmark features four distinct test suites: three evaluate the capability to train on human observations (with or without distractors) and test across varied distraction settings; and the fourth suite offers a calibrated evaluation for feature-extraction-based LfO approaches. RoboReel spans the full breadth of current LfO challenges while introducing aspirational suites for long-horizon understanding of observed tasks in cluttered environments.

observations has been of interest to the robot learning community with multiple surveys [6, 7] and state-of-art approaches [8, 9, 10, 11, 9, 12, 13]. Despite significant progress in this area, the field lacks benchmarks that enable systematic and reproducible evaluation of Learning from Observations (LfO) approaches. The absence of this reproducibility prevents systematic comparison across methods, as we are not capable of making any lateral (or apples to apples) comparison making it difficult to establish a reliable and reproducible state-of-the-art approach or draw meaningful conclusions about progress in the field.

Given the scaling and biological aspirations, there are significant recent works that focus on learning from human demonstration videos [10, 11, 9, 12, 13]. Broadly, these works either convert human videos to language instructions using VLMs [14, 15]; or learn some implicit representations from human videos to learn policies [16, 10, 17, 18, 19, 20, 21]; or utilize accurate 3D human pose annotations in a calibrated space for learning using imitation [22, 23] or in-context learning [24, 25, 26, 13]. These methods are trained and evaluated with different assumptions on the demonstration videos with varying train and test protocols. While there are significant efforts in creating datasets and benchmarks for learning from watching [27, 9, 26], these works do not provide a unified real human dataset with a calibrated and paired test environment with trajectories to train and evaluate LfO models. Crisply, standardized LfO evaluation faces two fundamental challenges: visual fidelity of the scene and the availability of a portable, calibrated environment to test robot performance.

To address this gap, we present RoboReel, a unified dataset and benchmark to train and evaluate LfO models. RoboReel leverages generative methods to reconstruct accurate 3D meshes of real-world objects, and pairs a calibrated real-world environment for human demonstrations with a matched simulation environment for an equitable evaluation. The benchmark presents multiple suites for testing a variety of learning algorithms from those that require wrist poses of humans in the observation to those that can account for distractions in the environment. The work provides over seven hours of paired and calibrated human data in ten table top tasks from multiple camera angels to measure the capability of LfO algorithms. We demonstrate the benchmark's utility by evaluating several SOTA approaches alongside Vision-Language-Action (VLA) baselines. This evaluation highlights the efficacy of the benchmark while uncovering the performance characteristics and underlying assumptions of different algorithms. Our major contributions are as follows:

1. We introduce a novel learning-from-observation benchmark, providing paired real-world human demonstration videos, robot trajectories, and evaluation environments on 10 tasks. To the best of our knowledge, our work is the first unified benchmark that provides paired training data and evaluation environment using real-world human demonstration videos for learning-from-observation which we will release on paper acceptance.
2. We train and evaluate over seven learning-from-observation models with varying representational choices.
3. We provide a detailed analysis on the effectiveness of the algorithms and existing gaps in SOTA LfO models.

## 2 Related Work

**Robotics Datasets, Benchmarks, and Simulators.** The computer vision community has long been interested in understanding human action videos [28, 29, 30, 31, 32, 33, 34, 35]. Unfortunately, this data does not allow accurate comparisons for the robots due to the lack of a paired environment to learn or evaluate actions or skills. We also have significant robotics datasets [36, 27, 37, 38], benchmarks [39, 40, 41, 42, 43], and simulators [44, 45, 46]. These works can contain paired language instructions with robot trajectories and/or a simulated environment. However, these works omit paired video demonstrations as an input modality, preventing them from evaluating whether an embodied agent can replicate actions seen from human video observations. RoboMME [39] includes some video-conditioning tasks to evaluate memory of embodied agents but it lacks any real world human data. To the best of our knowledge, RH20T [27] and the MimicDroid benchmark [26] are the closest to our work. RH20T [27] contains paired robot trajectories and human demonstration videos while lacking simulated evaluation environments to perform lateral comparisons. MimicDroid [26] provides paired robot demonstration videos and evaluation environments, but lacks real world human demonstration inputs. We address this gap by introducing paired real-world human videos and simulated environments to benchmark robotic learning from observation.

**Learning Robot Skills from Human Videos.** Previous LfO surveys have built taxonomies on existing methods by transfer pathways [6] or by algorithmic designs[7]. In this work, we categorize these methods by their representation choice for the input human observation video. As illustrated in Fig. 3, these representation choices fall into three broad categories: *(1) Extracting explicit language representations from videos* [14, 15, 47], *(2) Learning implicit latent representation from videos* [16, 10, 17, 21, 48], and *(3) Learning robot actions from keypoints with in-context-learning or imitation learning* [22, 24, 23, 26]. Explicit methods use VLMs to extract language instructions from human demonstration videos and then use planners or learned motion policies to complete tasks [14, 15, 47]. Implicit methods learn to encode the human videos into latent neural skill embeddings, and condition on these learned latent representations for policy improvement [16, 21, 10, 17]. Keypoint-based methods assume a common action space between the robot and the human and extract these actions using keypoints to perform imitation learning [22, 23, 25, 24]. To unify the evaluation of existing methods with diverse architectures and assumptions, we introduce a standardized benchmark to compare these model classes on identical tasks.

| Dataset Name | Real-world Human Vid. | Human Demo. W. Distractions | Multi-view Demo. | Paired Real-world Human Vid. Eval Env. | Env. w. Distractions | Multi-view Env. | Articulated Objects | Long Horizon |
|---|---|---|---|---|---|---|---|---|
| RoboCerebra [42] | ✗ | ✗ | ✗ | ✗ | ✓ | ✓ | ✓ | ✓ |
| LIBERO [40] | ✗ | ✗ | ✗ | ✗ | ✓ | ✓ | ✓ | ✓ |
| RoboCasa [46] | ✗ | ✗ | ✗ | ✗ | ✓ | ✓ | ✓ | ✓ |
| VLABench [49] | ✗ | ✗ | ✗ | ✗ | ✓ | ✓ | ✓ | ✓ |
| MimicGen [43] | ✗ | ✗ | ✗ | ✗ | ✓ | ✓ | ✓ | ✓ |
| EgoVerse [50] | ✓ | ✓ | ✗ | ✗ | ✗ | ✗ | ✓ | ✓ |
| Human2Robot [9] | ✓ | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ |
| RH20T [27] | ✓ | ✓ | ✓ | ✗ | ✗ | ✗ | ✓ | ✓ |
| MimicDroid [26] | ✗ | ✓ | ✗ | ✗ | ✓ | ✗ | ✓ | ✗ |
| **RoboReel (Ours)** | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

Table 1: **Benchmark and Dataset Comparison.**

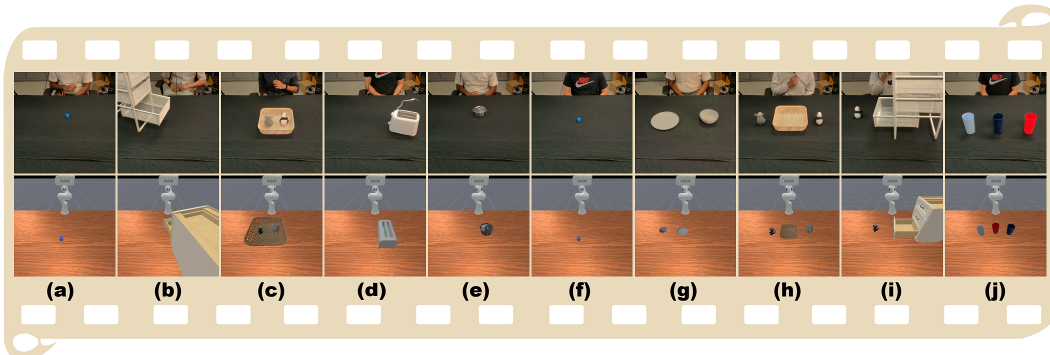


Figure 2: Tasks in the RoboReel Benchmark – (a) push cube, (b) close drawer, (c) empty basket, (d) press a toaster button, (e) open a lid, (f) pick a cube, (g) place a bowl in a plate, (h) place toys in a basket, (i) place a toy in a drawer and close the drawer, (j) stack three cups. These tasks vary in their horizon, types object interactions, and required tolerances during manipulation.

## 3 RoboReel - A Learning from Observation Benchmark

RoboReel provides a standard test bench to evaluate learning from observation (LfO) models. As shown in Table 1 while many benchmarks focus on scaling robot data with language conditioning, none include paired human observations for LfO benchmarking. The closest comparisons to our benchmark, MimicDroid [26], RHT20 [27] and Human2Robot [9] lack a critical requirement for equitable lateral comparisons of LfO methods – real world human data paired with simulation environment for systematic policy assessment, which we develop in the following section.

### 3.1 Research Categories addressed by the RoboReel

RoboReel features four carefully designed evaluation suites, each targeting a distinct training-evaluation paradigm. The **No Distraction Suite (ND)** and **With Distraction Suite (WD)**, evaluate manipulation under clean and distracted conditions, respectively. Distracted setting introduces non-task-related objects randomly sampled from a pre-generated asset pool, while the clean setting contains no such distractors. The **Evaluation Distraction Suite (ED)**, examines whether skills learned through observation are robust to observational domain shift introduced at evaluation time. Unlike the distracted setting, distraction objects are absent during training but are introduced in both the human demonstrations and the simulation environment at evaluation time. Finally we have a **Pose Defined Suite (PD)**, which examines whether keypoint annotations using a camera calibration setup benefit the LfO paradigm.

### 3.2 Task Curation

To evaluate the learning capability from observations, we design 10 tasks to test methods across three axes of variation: object type, task horizon, and manipulation precision. These tasks include placing a bowl in a plate, closing a drawer, emptying a basket, opening a lid, picking a cube, pressing a toaster button, pushing a cube, stacking three cups, placing toys in a basket, and placing a toy in a drawer and closing the drawer, as shown in Figure 2. The tasks encompass a wide variety of tabletop manipulation task set that the LfO literature typically evaluates [23, 21, 14]. We restrict our scope to goal-based tasks due to two key robotics bottlenecks: inaccurate soft-body simulations and inherent difficulty in validating non-goal-oriented tasks.

### 3.3 Human Demonstration Collection

To collect human demonstration videos, we construct a tabletop environment in our lab using 18 rigid-body objects, each with a corresponding digital twin in the simulation environment. The task videos are recorded from four perspectives: three exocentric views capturing the front, left, and right sides of the table using RealSense L515 cameras, and one egocentric view captured by a RealSense D435 camera mounted on a helmet worn by the task performer as shown in Figure 1. Both RGB and depth images are collected from all sensors, with timestamps provided for frame synchronization. We collect 100 demonstrations per task under clean and distracted conditions respectively, yielding a total of 2,000 human demonstration videos. Table 1 provides detailed dataset statistics.

### 3.4 Constructing a Corresponding Simulation Environment and Generating Paired data

We build our simulated digital twin test bench on ManiSkill [44] due to its native integration capabilities with PartNet-Mobility dataset [51, 52, 53], GPU-accelerated rendering with ray tracing support and gym-compatible environment interface that enables rapid prototyping of manipulation tasks with varied objects. We use the 7 DoF Franka Panda arm, chosen for its widespread adoption in real-world robot learning research. Articulated object assets for drawer and toasters are imported directly from Partnet-mobility dataset [51, 52, 53] which provides a rich library of articulated object models of various categories.

To bridge the visual gap between simulation and the real world, we reconstruct rigid objects as digital twins using SAM3D-objects [54]. These reconstructed meshes preserve high fidelity in color and shapem, and closely approximate the physical geometry of their real-world counterparts. We replicate the real-world camera configuration by using ArUco Markers to establish the camera coordinate relationships between the three fixed cameras and the table center. This configuration was then applied to the ManuSkill environment allowing camera poses that precisely match their real-world counterparts. This approach ensures visual consistency across domains for benchmarking models that require identical setup during training and testing phases.

To generate training data across tasks, we manually define success conditions that mirror the corresponding real-world configurations. For pick-and-place tasks, we use GraspGen [55] to generate plausible candidate grasps, which are then ranked according to constrains including joint movements and end-effector distance. We then use mplib [56] to automatically synthesize collision-free robot trajectories across five phases: pre-grasp, grasp, transport, pre-place, and drop. For tasks involving articulated objects such as drawers and doors, we define waypoints along the joint motion direction to guide the end-effector in manipulating the target link. Robot skills are labeled successful upon satisfaction of the predefined success goal condition.

## 4 Experiments

In this section, we systematically evaluate different learning-from-observation methods using RoboReel. We first describe our experiment setup and then analyze our results.

### 4.1 Baselines

We have categorized our baselines into three families as described in Sec. 2 and shown in Figure 3.

**Explicit Language Representation from Human Demos.** For this category, we evaluate SeeDo [14] by extracting the high-level plan generated by a pre-trained VLM planner. As SeeDo re-

| Model Family | Model | Bowl in Plate | Close Drawer | Empty Basket | Open Lid | Pick Cube | Press Toaster | Push Cube | Stack Cups | Toys in Basket | Toy in Drawer |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Explicit Representation** | SeeDo [14] | **71.3 ± 2.2** | N/A | **7.8 ± 1.9** | **54.1 ± 3.6** | N/A | N/A | N/A | 0.0 ± 0.0 | **18.7 ± 1.9** | N/A |
| | VLM + $\pi_{0.5}$ [57] | 1.7 ± 1.7 | **88.3 ± 3.3** | 1.7 ± 1.7 | 20.0 ± 2.9 | **6.7 ± 1.7** | **31.7 ± 3.3** | **18.3 ± 1.7** | 0.0 ± 0.0 | 0.0 ± 0.0 | **16.7 ± 4.4** |
| **Implicit Representation** | Vid. Cond. $\pi_{0.5}$ [57] (Uniform) | 31.7 ± 3.3 | 98.3 ± 1.7 | 18.3 ± 6.7 | **93.3 ± 4.4** | **55.0 ± 5.8** | 83.3 ± 3.3 | 78.3 ± 6.0 | **11.7 ± 3.3** | 20.0 ± 2.9 | 16.7 ± 7.3 |
| | Vid. Cond. $\pi_{0.5}$ [57] (Keyframes) | **46.7 ± 6.0** | **100.0 ± 0.0** | **20.0 ± 2.9** | 73.3 ± 1.7 | 23.3 ± 7.3 | **88.3 ± 6.7** | **100.0 ± 0.0** | 8.3 ± 3.3 | **21.7 ± 3.3** | **20.0 ± 7.6** |
| | Vid2Robot [16] | 1.7 ± 1.7 | 21.7 ± 3.3 | 3.3 ± 1.7 | 3.3 ± 3.3 | 1.7 ± 1.7 | 21.7 ± 6.0 | 5.0 ± 2.9 | 3.3 ± 1.7 | 0.0 ± 0.0 | 0.0 ± 0.0 |
| | UniSkill [21] | 0.0 ± 0.0 | 66.7 ± 8.8 | 0.0 ± 0.0 | 8.3 ± 1.7 | 10.0 ± 2.9 | 25.0 ± 10.0 | 0.0 ± 0.0 | 0.0 ± 0.0 | 0.0 ± 0.0 | 0.0 ± 0.0 |

Table 2: **Experiment results on the ND Suite(♠).** Models are grouped by transfer pathway. All models are trained on all tasks using 80 human demonstration videos without distractions and 100 robot trajectories collected in environments without distractions. This table reports results for the evaluation configuration where both the environment and demonstration contain no distractions.

| Model Family | Model | Bowl in Plate | Close Drawer | Empty Basket | Open Lid | Pick Cube | Press Toaster | Push Cube | Stack Cups | Toys in Basket | Toy in Drawer |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Explicit Representation** | SeeDo [14] | **66.3 ± 0.4** | N/A | 4.9 ± 2.0 | **37.8 ± 3.1** | N/A | N/A | N/A | 0.0 ± 0.0 | **11.1 ± 2.3** | N/A |
| | VLM + $\pi_{0.5}$ [57] | 30.0 ± 7.6 | 63.3 ± 4.4 | **13.3 ± 1.7** | 18.3 ± 4.4 | **5.0 ± 5.0** | 13.3 ± 1.7 | 15.0 ± 5.0 | 0.0 ± 0.0 | 1.7 ± 1.7 | 5.0 ± 2.9 |
| **Implicit Representation** | Vid. Cond. $\pi_{0.5}$ [57] (Uniform) | 86.7 ± 4.4 | **86.7 ± 3.3** | **25.0 ± 2.9** | 73.3 ± 10.1 | 13.3 ± 6.0 | 45.0 ± 7.6 | **71.7 ± 8.3** | 5.0 ± 0.0 | **5.0 ± 2.9** | **23.3 ± 3.3** |
| | Vid. Cond. $\pi_{0.5}$ [57] (Keyframes) | **88.3 ± 7.3** | 85.0 ± 5.0 | 21.7 ± 7.3 | **85.0 ± 2.9** | **48.3 ± 6.0** | **56.7 ± 6.0** | 21.7 ± 6.0 | **8.3 ± 4.4** | 3.3 ± 1.7 | 18.3 ± 1.7 |
| | Vid2Robot [16] | 6.7 ± 4.4 | 10.0 ± 5.8 | 3.3 ± 3.3 | 0.0 ± 0.0 | 0.0 ± 0.0 | 30.0 ± 5.0 | 8.3 ± 3.3 | 0.0 ± 0.0 | 0.0 ± 0.0 | 0.0 ± 0.0 |
| | UniSkill [21] | 0.0 ± 0.0 | 56.7 ± 13.6 | 10.0 ± 2.9 | 21.7 ± 6.7 | 1.7 ± 1.7 | 21.7 ± 7.3 | 8.3 ± 1.7 | 0.0 ± 0.0 | 0.0 ± 0.0 | 0.0 ± 0.0 |

Table 3: **Experiment results on Learning from Observation test suite with distractions(♡).** Models are grouped by transfer pathway. All models are trained on all tasks using 80 human demonstration videos and 100 robot trajectories collected in environments with distractions. This table reports results for the evaluation configuration where both the environment and demonstration contain distractions.

| Model Family | Model | Bowl in Plate | Close Drawer | Empty Basket | Open Lid | Pick Cube | Press Toaster | Push Cube | Stack Cups | Toys in Basket | Toy in Drawer |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Explicit Representation** | SeeDo [14] | **66.3 ± 0.4** | N/A | 4.9 ± 2.0 | **37.8 ± 3.1** | N/A | N/A | N/A | 0.0 ± 0.0 | **11.1 ± 2.3** | N/A |
| | VLM + $\pi_{0.5}$ [57] | 6.7 ± 3.3 | 61.7 ± 3.3 | 5.0 ± 5.0 | 6.7 ± 1.7 | 11.7 ± 4.4 | 15.0 ± 5.0 | 6.7 ± 1.7 | 0.0 ± 0.0 | 0.0 ± 0.0 | 0.0 ± 0.0 |
| **Implicit Representation** | Vid. Cond. $\pi_{0.5}$ [57] (Uniform) | 31.7 ± 4.4 | 58.3 ± 8.3 | 18.3 ± 4.4 | 61.7 ± 4.4 | 25.0 ± 5.8 | 15.0 ± 5.0 | 6.7 ± 4.4 | 5.0 ± 2.9 | 11.7 ± 4.4 | 6.7 ± 1.7 |
| | Vid. Cond. $\pi_{0.5}$ [57] (Keyframe) | 25.0 ± 5.0 | **70.0 ± 2.9** | **21.7 ± 4.4** | **63.3 ± 4.4** | **30.0 ± 5.0** | 26.7 ± 4.4 | **31.7 ± 3.3** | **11.7 ± 4.4** | **18.3 ± 3.3** | 6.7 ± 1.7 |
| | Vid2Robot [16] | 0.0 ± 0.0 | 23.3 ± 1.7 | 5.0 ± 2.9 | 0.0 ± 0.0 | 0.0 ± 0.0 | 23.3 ± 9.3 | 6.7 ± 3.3 | 0.0 ± 0.0 | 0.0 ± 0.0 | 0.0 ± 0.0 |
| | UniSkill [21] | 0.0 ± 0.0 | 63.3 ± 14.8 | 0.0 ± 0.0 | 18.3 ± 1.7 | 1.7 ± 1.7 | **28.3 ± 8.8** | 0.0 ± 0.0 | 0.0 ± 0.0 | 0.0 ± 0.0 | 0.0 ± 0.0 |

Table 4: **Experiment results on the ED test suit(♣).** Models are grouped by transfer pathway. All models are trained on all tasks using 80 human demonstration videos and 100 robot trajectories collected in environments without distraction. This table reports results for the evaluation configuration where both the environment and demonstration contain distractions.

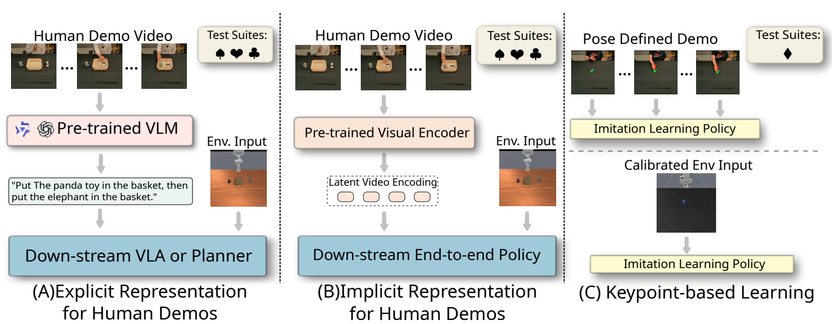


Figure 3: We categorize the baseline models we take into consideration by their representational choices to process human demonstration videos. Methods in (A) explicitly generate language instructions with VLMs given videos, and then use a downstream planner or VLA model to generate robot actions. Methods in (B) encode the human demonstrations into a learned latent space which is then used to guide a downstream neural network policy. Models in (C) directly perform imitation learning using human videos and their paired 3D hand poses in training, and directly rollout on an environment that has the same 3D action space in real world.

quires no training, we report its With Distraction results are equivalent to its Evaluation Distraction results. The action vocabulary here is also restricted to pick-and-place, so only the five pick-and-place tasks that fall within its capabilities are reported. We additionally implement a VLA-based explicit model using Qwen3-VL-4B [58] to extract language commands from demonstration videos, which are used to fine-tune $\pi_{0.5}$ [57]. To assess the quality of VLM-generated language guidance, we also train a $\pi_{0.5}$ baseline using expert language annotations.

**Learning Implicit Representation from Human Demos.** For this line of work, we train and test established baselines UniSkill [21] and Vid2Robot [16]. We additionally augment $\pi_{0.5}$ [57] to accept human video inputs by encoding demonstration frames into visual tokens, using two selection strategies: uniform frame sampling and a BPP-inspired keyframe selection method [59] that weights frames by proximity to the nearest keyframe.

**Pose Defined LfO** We evaluate PointPolicy [23] as our baseline for this category. As PointPolicy requires single-task training, we report results for five tasks here, with complete results provided in the Appendix A.

### 4.2 Results

**No Distraction Test Suite.** Table 2 shows the performance on the **ND** Suite. On this test suite, video-conditioned policies with implicit task representations perform strongest overall. Our two baseline variants of video-conditioning $\pi_{0.5}$ models outperform other baselines significantly across most tasks. The performance gap is especially obvious in the long-horizon tasks that involve multiple sub-tasks, including Empty Basket, Stack Cups, Toys in Basket, and Toy in Drawer. Despite having access to action primitives that can access the ground truth locations of the objects, SeeDo [14] does not perform well on multi-step pick-and-place tasks such as Empty Basket and Stack Cups.

**With Distraction Test Suite.** Table 3 describes the models' performance on the **WD** Suite. Similar to the **ND** suite, the two video-conditioned $\pi_{0.5}$ variants achieve the overall best performance. Although the **WD** suite evaluates models' performance with in-distribution settings, models from both families suffer a performance drop on the majority of the tasks.

**Evaluation Distraction Test Suite.** Table 4 describes the performance on the **ED** Suite. In this test suite, we find video-conditioned $\pi_{0.5}$ with keyframes achieves the overall best performance on most tasks. We also find a similar trend that implicit methods outperforming explicit methods on all tasks with significance. Because this **ED** suite is by design more challenging than the previous two task suites, most models cannot achieve a similar performance to the other task suites on most tasks.

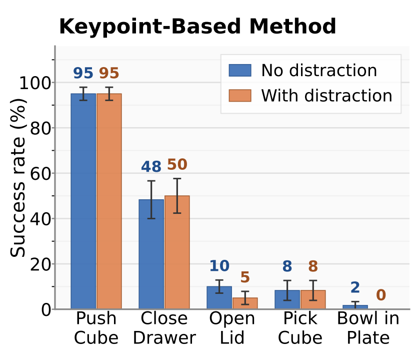


Figure 4: This figure presents the experiment results of PointPolicy on the keypoint-based test suite (◇) on five tasks. A different individual model is trained for each task.

**Pose Defined Test Suite.** In this test suite, we report the results of PointPolicy [23] on the five selected tasks in Fig. 4 with the rest of the results in the Appendix A. We find that PointPolicy is able to capture the behaviors of the human demonstrators from the keypoint data, achieving a high performance in the Push Cube and the Close Drawer task. Additionally, this method displays significant robustness towards distractions, achieving a similar performance under the environment with and without distraction. However, this method does not perform on the other three tasks that involve grasping, regardless of the distraction setting.

### 4.3 Analysis

We analyze our results by answering the following key research questions:

**Q1. Which model family performs the best? Across all ten tasks and every test suite, video-conditioned $\pi_{0.5}$ variants achieve the strongest overall performance**, which we attribute to the large-scale pretraining of VLA models that enables better correspondence between human video features and robot actions. While the difference between uniform-sampled and keyframe-based $\pi_{0.5}$ $\pi_{0.5}$ is marginal in the ND and WD suites, where training and evaluation share the same distraction conditions, the gap becomes stark in the ED suite, where **keyframe-based** $\pi_{0.5}$ outperforms uniform sampling in 7 out of 10 tasks. We attribute this to keyframes capturing task-relevant moments with significance in the demonstration, making the policy more robust to the visual domain shift introduced at evaluation time in the ED suite. In contrast, uniform sampling includes many redundant or uninformative frames that may act as noise when the evaluation distribution shifts.

**Q2. Does the task length affect the performance? Task horizon has a clear negative affect on success rate across all model families.** Long-horizon tasks yield lowest success rate across all suites. These tasks require the robot to execute multiple sequential sub-goals correctly, where failure at any intermediate step results to overvall task failure. In contrast, short-horizon tasks such as Push Cube, Pick Cube, and Close Drawer see substantially higher success rates across all model families. Even the strongest baseline, video conditioned $\pi_{0.5}$ variants struggles on long horizon tasks with success remaining below 25%.

**Q3. Do VLAs benefit more from explicit natural language guidance?** We hypothesized that the internet-scale pretraining enables VLMs to understand simple task demonstration videos, and can correctly summarize the natural language instructions regardless of the distraction objects. Additionally, we expected VLAs to perform better with the language modality as these VLAs are extensively pre-trained with instruction following. However, **from our experiment results across three test suites, we find that implicit representation consistently outperforms explicit representation.** Although both VLMs and VLAs go through internet-scale pre-training with image and text data, we hypothesize the extra exposure to the robot trajectory data helps VLAs to overcome VLMs' weakness in visual-temporal reasoning [60].

**Q4. Can models learn tasks requiring lower tolerances manipulation?** We identify Press Toaster and Stack Cups as two tasks have low tolerances between objects during manipulation. For Press Toaster, models achieve relatively good performance in the ND suite with $\pi_{0.5}$ variant performing over 88%, but performance degrades consistently from the ND suite to the WD suite and further to the ED suite across all model families. We attribute this degradation to the small target area of the toaster button, where distractor objects in the scene introduce visual clutter that confuses the vision encoder, making it harder to localize the precise interaction point. For Stack Cups, performance is uniformly low across all suites and all models, as this task not only requires precise placement but is also long-horizon, compounding the difficulty.

**Q5. Do calibrated keypoints help in learning from observations?** From our experiments with PointPolicy [23], we find that while keypoint annotations help the model learn the overall trajectory behavior, this does not translate to consistent performance gains across tasks. As shown in Figure 4, PointPolicy achieves strong results on tasks such as Push Cube and Close Drawer, but performance drops sharply on tasks requiring precise grasping such as Pick Cube, Open Lid, and Bowl in Plate. We attribute this to the inherent difficulty of deriving grasp configurations purely from keypoints, especially on objects like bowls and handles. Precise grasp specifications without images is causing many grasp failures in our observations.

**Q6. What are the fundamental limitations of current LfO methods?** Our analysis across five research questions reveals consistent gaps in the current LfO paradigm. First, all model families struggle with long-horizon tasks, where compounding errors across sequential sub-goals remain a critical bottleneck. Second, precise manipulation tasks are hard to learn, especially in cluttered environment. Finally, while large-scale VLA pretraining provides a strong foundation, long-horizon tasks and tasks requiring precise manipulation remain unsolved across all evaluated methods.

## 5 Limitations

A significant limitation of our approach is the use of goal based tasks. Humans can perform non-Markovian and dynamic skills whose goals cannot be tested at a single time point, but this is a wider challenge within robotics. We also do not have any soft objects because their physics are harder to simulate. Another major limitation of our approach is the lack of bimanual tasks. Given most approaches in LfO work on single arm domains we restricted ourselves to tasks that use a single arm. We note that the presence of multiple arms would make action generation significantly challenging for existing models.

## 6 Conclusion

In conclusion, we contribute RoboReel, a benchmark to evaluate existing Learning-from-Observation frameworks with real-world human demonstrations. We manually curate $10$ tasks, collect $2,000$ real-world human demonstrations from multiple angles, and develop paired evaluation environments in the simulation. Using the $4$ task suites from RoboReel, we study over seven SOTA models from various model families, and analyze their performance on learning from observation under different settings. To the best of our knowledge, RoboReel is the first benchmark that introduces a shared evaluation protocol for these different model families. We plan to continuously develop RoboReel to increase the scale of the data and grow the variety of the tasks over time.

## References


[1] A. Bandura, D. M. Ross, and S. A. Ross. Vicarious reinforcement and imitative learning. *Journal of abnormal psychology*, 67:601–7, 1963. URL https://api.semanticscholar.org/CorpusID:28737713.

[2] M. J. Fryling, C. Johnston, and L. J. Hayes. Understanding observational learning: An interbehavioral approach. *The Analysis of Verbal Behavior*, 27:191–203, 2011. URL https://api.semanticscholar.org/CorpusID:19109058.

[3] L. M. Hopper, S. P. Lambeth, S. J. Schapiro, and A. Whiten. Observational learning in chimpanzees and children studied through 'ghost' conditions. *Proceedings of the Royal Society B: Biological Sciences*, 275:835 – 840, 2008. URL https://api.semanticscholar.org/CorpusID:11613780.

[4] D. Mersmann, M. Tomasello, J. Call, J. Kaminski, and M. Taborsky. Simple mechanisms can explain social learning in domestic dogs (canis familiaris). *Ethology*, 117:675–690, 2011. URL https://api.semanticscholar.org/CorpusID:1302283.

[5] G. B. Biederman, H. A. Robertson, and M. Vanayan. Observational learning of two visual discriminations by pigeons: a within-subjects design. *Journal of the experimental analysis of behavior*, 46 1:45–9, 1986. URL https://api.semanticscholar.org/CorpusID:38606737.

[6] J. Ma, E. Zhang, H. Yang, D. Li, C. Xu, G. Wang, and H. Wang. Robot learning from human videos: A survey, 2026. URL https://arxiv.org/abs/2604.27621.

[7] R. Burnwal, H. Mehta, N. P. Bhatt, and B. Ravindran. Learning from observation: A survey of recent advances. *ArXiv*, abs/2509.19379, 2025. URL https://api.semanticscholar.org/CorpusID:281505412.

[8] Y. Yang, Y. Li, C. Fermüller, and Y. Aloimonos. Robot learning manipulation action plans by " watching " unconstrained videos from the world wide web. 01 2015.

[9] S. Xie, H. Cao, Z. Weng, Z. Xing, H. Chen, S. Shen, J. Leng, Z. Wu, and Y.-G. Jiang. Human2robot: Learning robot actions from paired human-robot videos, 2025. URL https://arxiv.org/abs/2502.16587.

[10] E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch, S. Levine, and C. Finn. Bc-z: Zero-shot task generalization with robotic imitation learning, 2022. URL https://arxiv.org/abs/2202.02005.

[11] G. Li, Y. Lyu, Z. Liu, C. Hou, J. Zhang, and S. Zhang. H2r: A human-to-robot data augmentation for robot pre-training from videos, 2026. URL https://arxiv.org/abs/2505.11920.

[12] M. Xu, Z. Xu, C. Chi, M. Veloso, and S. Song. Xskill: Cross embodiment skill discovery, 2023. URL https://arxiv.org/abs/2307.09955.

[13] X. Zhu, Y. Liu, H. Li, and J. Chen. Learning generalizable robot policy with human demonstration video as a prompt, 2025. URL https://arxiv.org/abs/2505.20795.

[14] B. Wang, J. Zhang, S. Dong, I. Fang, and C. Feng. Vlm see, robot do: Human demo video to robot action plan via vision language model, 2025. URL https://arxiv.org/abs/2410.08792.

[15] X. Lin, S. Lian, B. Yu, R. Yang, Z. Shen, C. Wu, Y. Miao, Y. Jin, Y. Shi, J. He, C. Huang, B. Cheng, and K. Chen. Physbrain: Human egocentric data as a bridge from vision language models to physical intelligence, 2026. URL https://arxiv.org/abs/2512.16793.

[16] V. Jain, M. Attarian, N. J. Joshi, A. Wahid, D. Driess, Q. Vuong, P. R. Sanketi, P. Sermanet, S. Welker, C. Chan, I. Gilitschenski, Y. Bisk, and D. Dwibedi. Vid2robot: End-to-end video-conditioned policy learning with cross-attention transformers, 2024. URL https://arxiv.org/abs/2403.12943.

[17] K. Kedia, P. Dan, A. Chao, M. A. Pace, and S. Choudhury. One-shot imitation under mismatched execution, 2025. URL https://arxiv.org/abs/2409.06615.

[18] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3m: A universal visual representation for robot manipulation, 2022. URL https://arxiv.org/abs/2203.12601.

[19] T. Xiao, I. Radosavovic, T. Darrell, and J. Malik. Masked visual pre-training for motor control, 2022. URL https://arxiv.org/abs/2203.06173.

[20] J. Pai, L. Achenbach, V. Montesinos, B. Forrai, O. Mees, and E. Nava. mimic-video: Video-action models for generalizable robot control beyond vlas, 2025. URL https://arxiv.org/abs/2512.15692.

[21] H. Kim, J. Kang, H. Kang, M. Cho, S. J. Kim, and Y. Lee. Uniskill: Imitating human videos via cross-embodiment skill representations, 2025. URL https://arxiv.org/abs/2505.08787.

[22] J. Ren, P. Sundaresan, D. Sadigh, S. Choudhury, and J. Bohg. Motion tracks: A unified representation for human-robot transfer in few-shot imitation learning, 2025. URL https://arxiv.org/abs/2501.06994.

[23] S. Haldar and L. Pinto. Point policy: Unifying observations and actions with key points for robot manipulation, 2025. URL https://arxiv.org/abs/2502.20391.

[24] G. Papagiannis, N. D. Palo, P. Vitiello, and E. Johns. R+x: Retrieval and execution from everyday human videos, 2025. URL https://arxiv.org/abs/2407.12957.

[25] N. D. Palo and E. Johns. Keypoint action tokens enable in-context imitation learning in robotics, 2024. URL https://arxiv.org/abs/2403.19578.

[26] R. Shah, S. Liu, Q. Wang, Z. Jiang, S. Kumar, M. Seo, R. Martín-Martín, and Y. Zhu. Mimicdroid: In-context learning for humanoid manipulation from human play videos. 2026.

[27] H.-S. Fang, H. Fang, Z. Tang, J. Liu, C. Wang, J. Wang, H. Zhu, and C. Lu. Rh20t: A comprehensive robotic dataset for learning diverse skills in one-shot, 2023. URL https://arxiv.org/abs/2307.00595.

[28] K. Soomro, A. R. Zamir, and M. Shah. Ucf101: A dataset of 101 human actions classes from videos in the wild, 2012. URL https://arxiv.org/abs/1212.0402.

[29] R. Goyal, S. E. Kahou, V. Michalski, J. Materzynska, S. Westphal, H. Kim, V. Haenel, I. Fruend, P. Yianilos, M. Mueller-Freitag, F. Hoppe, C. Thurau, I. Bax, and R. Memisevic. The "something something" video database for learning and evaluating visual common sense. pages 5843–5851, 10 2017. doi:10.1109/ICCV.2017.622.

[30] F. C. Heilbron, V. Escorcia, B. Ghanem, and J. C. Niebles. Activitynet: A large-scale video benchmark for human activity understanding. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 961–970, 2015. doi:10.1109/CVPR.2015.7298698.

[31] A. Miech, D. Zhukov, J.-B. Alayrac, M. Tapaswi, I. Laptev, and J. Sivic. Howto100m: Learning a text-video embedding by watching hundred million narrated video clips, 2019. URL https://arxiv.org/abs/1906.03327.

[32] E. Garrote, T. Serre, H. Jhuang, H. Kuehne, and T. Poggio. HMDB: A large video database for human motion recognition . In *2011 IEEE International Conference on Computer Vision (ICCV 2011)*, pages 2556–2563, Los Alamitos, CA, USA, Nov. 2011. IEEE Computer Society. doi:10.1109/ICCV.2011.6126543. URL https://doi.ieeecomputersociety.org/10.1109/ICCV.2011.6126543.

[33] F. Sener, D. Chatterjee, D. Shelepov, K. He, D. Singhania, R. Wang, and A. Yao. Assembly101: A large-scale multi-view video dataset for understanding procedural activities, 2022. URL https://arxiv.org/abs/2203.14712.

[34] K. Grauman, A. Westbury, L. Torresani, K. Kitani, J. Malik, T. Afouras, K. Ashutosh, V. Baiyya, S. Bansal, B. Boote, E. Byrne, Z. Chavis, J. Chen, F. Cheng, F.-J. Chu, S. Crane, A. Dasgupta, J. Dong, M. Escobar, C. Forigua, A. Gebreselasie, S. Haresh, J. Huang, M. M. Islam, S. Jain, R. Khirodkar, D. Kukreja, K. J. Liang, J.-W. Liu, S. Majumder, Y. Mao, M. Martin, E. Mavroudi, T. Nagarajan, F. Ragusa, S. K. Ramakrishnan, L. Seminara, A. Somayazulu, Y. Song, S. Su, Z. Xue, E. Zhang, J. Zhang, A. Castillo, C. Chen, X. Fu, R. Furuta, C. Gonzalez, P. Gupta, J. Hu, Y. Huang, Y. Huang, W. Khoo, A. Kumar, R. Kuo, S. Lakhavani, M. Liu, M. Luo, Z. Luo, B. Meredith, A. Miller, O. Oguntola, X. Pan, P. Peng, S. Pramanick, M. Ramazanova, F. Ryan, W. Shan, K. Somasundaram, C. Song, A. Southerland, M. Tateno, H. Wang, Y. Wang, T. Yagi, M. Yan, X. Yang, Z. Yu, S. C. Zha, C. Zhao, Z. Zhao, Z. Zhu, J. Zhuo, P. Arbelaez, G. Bertasius, D. Crandall, D. Damen, J. Engel, G. M. Farinella, A. Furnari, B. Ghanem, J. Hoffman, C. V. Jawahar, R. Newcombe, H. S. Park, J. M. Rehg, Y. Sato, M. Savva, J. Shi, M. Z. Shou, and M. Wray. Ego-exo4d: Understanding skilled human activity from first- and third-person perspectives, 2024. URL https://arxiv.org/abs/2311.18259.

[35] D. Damen, H. Doughty, G. M. Farinella, S. Fidler, A. Furnari, E. Kazakos, D. Moltisanti, J. Munro, T. Perrett, W. Price, and M. Wray. Scaling egocentric vision: The epic-kitchens dataset, 2018. URL https://arxiv.org/abs/1804.02748.

[36] O. X. E. Collaboration. Open x-embodiment: Robotic learning datasets and rt-x models, 2025. URL https://arxiv.org/abs/2310.08864.

[37] H. Walke, K. Black, A. Lee, M. J. Kim, M. Du, C. Zheng, T. Zhao, P. Hansen-Estruch, Q. Vuong, A. He, V. Myers, K. Fang, C. Finn, and S. Levine. Bridgedata v2: A dataset for robot learning at scale, 2024. URL https://arxiv.org/abs/2308.12952.

[38] D. D. Team. Droid: A large-scale in-the-wild robot manipulation dataset, 2025. URL https://arxiv.org/abs/2403.12945.

[39] Y. Dai, H. Fu, J. Lee, Y. Liu, H. Zhang, J. Yang, C. Finn, N. Fazeli, and J. Chai. Robomme: Benchmarking and understanding memory for robotic generalist policies, 2026. URL https://arxiv.org/abs/2603.04639.

[40] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning, 2023. URL https://arxiv.org/abs/2306.03310.

[41] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison. Rlbench: The robot learning benchmark & learning environment, 2019. URL https://arxiv.org/abs/1909.12271.

[42] S. Han, B. Qiu, Y. Liao, S. Huang, C. Gao, S. Yan, and S. Liu. Robocerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation, 2025. URL https://arxiv.org/abs/2506.06677.

[43] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations, 2023. URL https://arxiv.org/abs/2310.17596.

[44] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T. kai Chan, Y. Gao, X. Li, T. Mu, N. Xiao, A. Gurha, V. N. Rajesh, Y. W. Choi, Y.-R. Chen, Z. Huang, R. Calandra, R. Chen, S. Luo, and H. Su. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai, 2025. URL https://arxiv.org/abs/2410.00425.

[45] H. Geng, F. Wang, S. Wei, Y. Li, B. Wang, B. An, C. T. Cheng, H. Lou, P. Li, Y.-J. Wang, Y. Liang, D. Goetting, C. Xu, H. Chen, Y. Qian, Y. Geng, J. Mao, W. Wan, M. Zhang, J. Lyu, S. Zhao, J. Zhang, J. Zhang, C. Zhao, H. Lu, Y. Ding, R. Gong, Y. Wang, Y. Kuang, R. Wu, B. Jia, C. Sferrazza, H. Dong, S. Huang, Y. Wang, J. Malik, and P. Abbeel. Roboverse: Towards a unified platform, dataset and benchmark for scalable and generalizable robot learning, 2025. URL https://arxiv.org/abs/2504.18904.

[46] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots, 2024. URL https://arxiv.org/abs/2406.02523.

[47] C. Hori, M. Kambara, K. Sugiura, K. Ota, S. Khurana, S. Jain, R. Corcodel, D. Jha, D. Romeres, and J. Le Roux. Interactive robot action replanning using multimodal llm trained from human demonstration videos. In *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5, 2025. doi: 10.1109/ICASSP49660.2025.10887717.

[48] A. Gupta, W. Gu, O. Patil, J. K. Lee, and N. Gopalan. Pokenet: Learning kinematic models of articulated objects from human observations, 2026. URL https://arxiv.org/abs/2602.02741.

[49] S. Zhang, Z. Xu, P. Liu, X. Yu, Y. Li, Q. Gao, Z. Fei, Z. Yin, Z. Wu, Y.-G. Jiang, and X. Qiu. Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks, 2024. URL https://arxiv.org/abs/2412.18194.

[50] R. Punamiya, S. Kareer, Z. Liu, J. Citron, R.-Z. Qiu, X. Cai, A. Gavryushin, J. Chen, D. Liconti, L. Y. Zhu, P. Aphiwetsa, B. Li, A. Cheluva, P. Kuppili, Y. Liu, D. Patel, A. Gao, H.-Y. Chung, R. Co, R. Zbizika, J. Liu, X. Xu, H. Xiong, G. Chen, S. Oliani, C. Yang, X. Wang, J. Fort, R. Newcombe, J. Gao, J. Chong, G. Matsuda, A. Doriwala, M. Pollefeys, R. Katzschmann, X. Wang, S. Song, J. Hoffman, and D. Xu. Egoverse: An egocentric human dataset for robot learning from around the world, 2026. URL https://arxiv.org/abs/2604.07607.

[51] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, et al. Shapenet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*, 2015.

[52] F. Xiang, Y. Qin, K. Mo, Y. Xia, H. Zhu, F. Liu, M. Liu, H. Jiang, Y. Yuan, H. Wang, L. Yi, A. X. Chang, L. J. Guibas, and H. Su. SAPIEN: A simulated part-based interactive environment. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[53] K. Mo, S. Zhu, A. X. Chang, L. Yi, S. Tripathi, L. J. Guibas, and H. Su. PartNet: A large-scale benchmark for fine-grained and hierarchical part-level 3D object understanding. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[54] S. D. Team, X. Chen, F.-J. Chu, P. Gleize, K. J. Liang, A. Sax, H. Tang, W. Wang, M. Guo, T. Hardin, X. Li, A. Lin, J. Liu, Z. Ma, A. Sagar, B. Song, X. Wang, J. Yang, B. Zhang, P. Dollár, G. Gkioxari, M. Feiszli, and J. Malik. Sam 3d: 3dfy anything in images, 2025. URL https://arxiv.org/abs/2511.16624.

[55] A. Murali, B. Sundaralingam, Y.-W. Chao, J. Yamada, W. Yuan, M. Carlson, F. Ramos, S. Birchfield, D. Fox, and C. Eppner. Graspgen: A diffusion-based framework for 6-dof grasping with on-generator training. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2026. URL https://arxiv.org/abs/2507.13097.

[56] R. K. Guo, X. Lin, M. Liu, J. Gu, and H. Su. MPlib: a Lightweight Motion Planning Library. https://github.com/haosulab/MPlib. Lightweight Python package for motion planning. Documentation: https://motion-planning-lib.readthedocs.io/latest/.

[57] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL https://arxiv.org/abs/2504.16054.

[58] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, W. Ge, Z. Guo, Q. Huang, J. Huang, F. Huang, B. Hui, S. Jiang, Z. Li, M. Li, M. Li, K. Li, Z. Lin, J. Lin, X. Liu, J. Liu, C. Liu, Y. Liu, D. Liu, S. Liu, D. Lu, R. Luo, C. Lv, R. Men, L. Meng, X. Ren, X. Ren, S. Song, Y. Sun, J. Tang, J. Tu, J. Wan, P. Wang, P. Wang, Q. Wang, Y. Wang, T. Xie, Y. Xu, H. Xu, J. Xu, Z. Yang, M. Yang, J. Yang, A. Yang, B. Yu, F. Zhang, H. Zhang, X. Zhang, B. Zheng, H. Zhong, J. Zhou, F. Zhou, J. Zhou, Y. Zhu, and K. Zhu. Qwen3-vl technical report, 2025. URL https://arxiv.org/abs/2511.21631.

[59] M. S. Mark, J. Liang, M. Attarian, C. Fu, D. Dwibedi, D. Shah, and A. Kumar. Bpp: Long-context robot imitation learning by focusing on key history frames, 2026. URL https://arxiv.org/abs/2602.15010.

[60] M. F. Imam, C. Lyu, and A. F. Aji. Can multimodal llms do visual temporal understanding and reasoning? the answer is no!, 2025. URL https://arxiv.org/abs/2501.10674.

[61] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, A. Ananthram, A. Nathan, A. Luo, A. Helyar, A. Madry, A. Efremov, A. Spyra, A. Baker-Whitcomb, A. Beutel, A. Karpenko, A. Makelov, A. Neitz, A. Wei, A. Barr, A. Kirchmeyer, A. Ivanov, A. Christakis, A. Gillespie, A. Tam, A. Bennett, A. Wan, A. Huang, A. M. Sandjideh, A. Yang, A. Kumar, A. Saraiva, A. Vallone, A. Gheorghe, A. G. Garcia, A. Braunstein, A. Liu, A. Schmidt, A. Mereskin, A. Mishchenko, A. Applebaum, A. Rogerson, A. Rajan, A. Wei, A. Kotha, A. Srivastava, A. Agrawal, A. Vijayvergiya, A. Tyra, A. Nair, A. Nayak, B. Eggers, B. Ji, B. Hoover, B. Chen, B. Chen, B. Barak, B. Minaiev, B. Hao, B. Baker, B. Lightcap, B. McKinzie, B. Wang, B. Quinn, B. Fioca, B. Hsu, B. Yang, B. Yu, B. Zhang, B. Brenner, C. R. Zetino, C. Raymond, C. Lugaresi, C. Paz, C. Hudson, C. Whitney, C. Li, C. Chen, C. Cole, C. Voss, C. Ding, C. Shen, C. Huang, C. Colby, C. Hallacy, C. Koch, C. Lu, C. Kaplan, C. Kim, C. Minott-Henriques, C. Frey, C. Yu, C. Czarnecki, C. Reid, C. Wei, C. Decareaux, C. Scheau, C. Zhang, C. Forbes, D. Tang, D. Goldberg, D. Roberts, D. Palmie, D. Kappler, D. Levine, D. Wright, D. Leo, D. Lin, D. Robinson, D. Grabb, D. Chen, D. Lim, D. Salama, D. Bhattacharjee, D. Tsipras, D. Li, D. Yu, D. Strouse, D. Williams, D. Hunn, E. Bayes, E. Arbus, E. Akyurek, E. Y. Le, E. Widmann, E. Yani, E. Proehl, E. Sert, E. Cheung, E. Schwartz, E. Han, E. Jiang, E. Mitchell, E. Sigler, E. Wallace, E. Ritter, E. Kavanaugh, E. Mays, E. Nikishin, F. Li, F. P. Such, F. de Avila Belbute Peres, F. Raso, F. Bekerman, F. Tsimpourlas, F. Chantzis, F. Song, F. Zhang, G. Raila, G. McGrath, G. Briggs, G. Yang, G. Parascandolo, G. Chabot, G. Kim, G. Zhao, G. Valiant, G. Leclerc, H. Salman, H. Wang, H. Sheng, H. Jiang, H. Wang, H. Jin, H. Sikchi, H. Schmidt, H. Aspegren, H. Chen, H. Qiu, H. Lightman, I. Covert, I. Kivlichan, I. Silber, I. Sohl, I. Hammoud, I. Clavera, I. Lan, I. Akkaya, I. Kostrikov, I. Kofman, I. Etinger, I. Singal, J. Hehir, J. Huh, J. Pan, J. Wilczynski, J. Pachocki, J. Lee, J. Quinn, J. Kiros, J. Kalra, J. Samaroo, J. Wang, J. Wolfe, J. Chen, J. Wang,

J. Harb, J. Han, J. Wang, J. Zhao, J. Chen, J. Yang, J. Tworek, J. Chand, J. Landon, J. Liang, J. Lin, J. Liu, J. Wang, J. Tang, J. Yin, J. Jang, J. Morris, J. Flynn, J. Ferstad, J. Heidecke, J. Fishbein, J. Hallman, J. Grant, J. Chien, J. Gordon, J. Park, J. Liss, J. Kraaijeveld, J. Guay, J. Mo, J. Lawson, J. McGrath, J. Vendrow, J. Jiao, J. Lee, J. Steele, J. Wang, J. Mao, K. Chen, K. Hayashi, K. Xiao, K. Salahi, K. Wu, K. Sekhri, K. Sharma, K. Singhal, K. Li, K. Nguyen, K. Gu-Lemberg, K. King, K. Liu, K. Stone, K. Yu, K. Ying, K. Georgiev, K. Lim, K. Tirumala, K. Miller, L. Ahmad, L. Lv, L. Clare, L. Fauconnet, L. Itow, L. Yang, L. Romaniuk, L. Anise, L. Byron, L. Pathak, L. Maksin, L. Lo, L. Ho, L. Jing, L. Wu, L. Xiong, L. Mamitsuka, L. Yang, L. McCallum, L. Held, L. Bourgeois, L. Engstrom, L. Kuhn, L. Feuvrier, L. Zhang, L. Switzer, L. Kondraciuk, L. Kaiser, M. Joglekar, M. Singh, M. Shah, M. Stratta, M. Williams, M. Chen, M. Sun, M. Cayton, M. Li, M. Zhang, M. Aljubeh, M. Nichols, M. Haines, M. Schwarzer, M. Gupta, M. Shah, M. Y. Guan, M. Huang, M. Dong, M. Wang, M. Glaese, M. Carroll, M. Lampe, M. Malek, M. Sharman, M. Zhang, M. Wang, M. Pokrass, M. Florian, M. Pavlov, M. Wang, M. Chen, M. Wang, M. Feng, M. Bavarian, M. Lin, M. Abdool, M. Rohaninejad, N. Soto, N. Staudacher, N. LaFontaine, N. Marwell, N. Liu, N. Preston, N. Turley, N. Ansman, N. Blades, N. Pancha, N. Mikhaylin, N. Felix, N. Handa, N. Rai, N. Keskar, N. Brown, O. Nachum, O. Boiko, O. Murk, O. Watkins, O. Gleeson, P. Mishkin, P. Lesiewicz, P. Baltescu, P. Belov, P. Zhokhov, P. Pronin, P. Guo, P. Thacker, Q. Liu, Q. Yuan, Q. Liu, R. Dias, R. Puckett, R. Arora, R. T. Mullapudi, R. Gaon, R. Miyara, R. Song, R. Aggarwal, R. Marsan, R. Yemiru, R. Xiong, R. Kshirsagar, R. Nuttall, R. Tsiupa, R. Eldan, R. Wang, R. James, R. Ziv, R. Shu, R. Nigmatullin, S. Jain, S. Talaie, S. Altman, S. Arnesen, S. Toizer, S. Toyer, S. Miserendino, S. Agarwal, S. Yoo, S. Heon, S. Ethersmith, S. Grove, S. Taylor, S. Bubeck, S. Banesiu, S. Amdo, S. Zhao, S. Wu, S. Santurkar, S. Zhao, S. R. Chaudhuri, S. Krishnaswamy, Shuaiqi, Xia, S. Cheng, S. Anadkat, S. P. Fishman, S. Tobin, S. Fu, S. Jain, S. Mei, S. Egoian, S. Kim, S. Golden, S. Mah, S. Lin, S. Imm, S. Sharpe, S. Yadlowsky, S. Choudhry, S. Eum, S. Sanjeev, T. Khan, T. Stramer, T. Wang, T. Xin, T. Gogineni, T. Christianson, T. Sanders, T. Patwardhan, T. Degry, T. Shadwell, T. Fu, T. Gao, T. Garipov, T. Sriskandarajah, T. Sherbakov, T. Korbak, T. Kaftan, T. Hiratsuka, T. Wang, T. Song, T. Zhao, T. Peterson, V. Kharitonov, V. Chernova, V. Kosaraju, V. Kuo, V. Pong, V. Verma, V. Petrov, W. Jiang, W. Zhang, W. Zhou, W. Xie, W. Zhan, W. McCabe, W. DePue, W. Ellsworth, W. Bain, W. Thompson, X. Chen, X. Qi, X. Xiang, X. Shi, Y. Dubois, Y. Yu, Y. Khakbaz, Y. Wu, Y. Qian, Y. T. Lee, Y. Chen, Y. Zhang, Y. Xiong, Y. Tian, Y. Cha, Y. Bai, Y. Yang, Y. Yuan, Y. Li, Y. Zhang, Y. Yang, Y. Jin, Y. Jiang, Y. Wang, Y. Wang, Y. Liu, Z. Stubenvoll, Z. Dou, Z. Wu, and Z. Wang. Openai gpt-5 system card, 2026. URL https://arxiv.org/abs/2601.03267.

[62] L. Tang, M. Jia, Q. Wang, C. P. Phoo, and B. Hariharan. Emergent correspondence from image diffusion. *Advances in neural information processing systems*, 36:1363–1389, 2023.

[63] N. Karaev, I. Rocco, B. Graham, N. Neverova, A. Vedaldi, and C. Rupprecht. Cotracker: It is better to track together. In *European conference on computer vision*, pages 18–35. Springer, 2024.

[64] W. Kay, J. Carreira, K. Simonyan, B. Zhang, C. Hillier, S. Vijayanarasimhan, F. Viola, T. Green, T. Back, P. Natsev, M. Suleyman, and A. Zisserman. The kinetics human action video dataset, 2017. URL https://arxiv.org/abs/1705.06950.

[65] C. Gu, C. Sun, D. A. Ross, C. Vondrick, C. Pantofaru, Y. Li, S. Vijayanarasimhan, G. Toderici, S. Ricco, R. Sukthankar, C. Schmid, and J. Malik. Ava: A video dataset of spatio-temporally localized atomic visual actions, 2018. URL https://arxiv.org/abs/1705.08421.

[66] H. Li, I. Zhang, R. Ouyang, X. Wang, Z. Zhu, Z. Yang, Z. Zhang, B. Wang, C. Ni, W. Qin, X. Chen, Y. Ye, G. Huang, Z. Song, and X. Wang. Mimicdreamer: Aligning human and robot demonstrations for scalable vla training, 2025. URL https://arxiv.org/abs/2509.22199.

[67] S. Ye, J. Jang, B. Jeon, S. Joo, J. Yang, B. Peng, A. Mandlekar, R. Tan, Y.-W. Chao, B. Y. Lin, L. Liden, K. Lee, J. Gao, L. Zettlemoyer, D. Fox, and M. Seo. Latent action pretraining from videos, 2025. URL https://arxiv.org/abs/2410.11758.

[68] Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li. Univla: Learning to act anywhere with task-centric latent actions, 2025. URL https://arxiv.org/abs/2505.06111.

## Contributions

**Weiwei Gu.** Implemented the models, evaluated baselines, organized the real-world data collection, wrote the appendix, contributed to writing the paper and supported the real robot experiments.

**Anmol Gupta.** Primarily developed the benchmark, designed the simulated tasks, developed the simulation data-collection pipeline, supported real-world data collection, and served as the primary writer of the paper.

**Anant Sah.** Conducted the real-world robot experiments, contributed to real-world data collection, and supported the implementation of baseline methods.

**Ryan Varghese.** Collected the real-world human demonstration data.

**Lalitha Shreya Vanam.** Collected the real-world human demonstration data.

**Prabhath Adireddi.** Collected the real-world human demonstration data and assisted with collecting the real-world robot data.

**Peter Karkus.** Provided guidance on the problem formulation and experimental design.

**Nakul Gopalan.** Supervised the project and contributed to writing the paper.

## A Additional Results for PointPolicy

Across the five extended tasks (empty basket, stack cups, toaster, toy drawer close, and toys in basket), the trained Point Policy achieves 0% success under the simulator's native success criterion. The policy nonetheless exhibits directionally correct behavior on several tasks, approaching the toys in toys in basket and reaching the drawer in toy drawer close. Two factors contribute to this performance gap.

First, the extended tasks have substantially longer horizons and require multiple sequential subskills: approach, grasp, lift, transport, and release. By contrast, the previously evaluated tasks were largely single-stage motions such as a planar push or a single grasp. The policy reliably reproduces the initial approach phase observed in the demonstrations but frequently fails to execute the subsequent manipulation stages needed to satisfy the environment's success condition.

Second, Point Policy operates on 3D keypoint coordinates and learns geometric relationships between channels rather than visual features. Although our sim-side and training-side labels follow the same ordering, the underlying keypoints do not correspond to exactly the same physical features. Small drift in where each channel is anchored (a few millimeters to a couple of centimeters between train and sim) shifts the relational offsets the policy learned. Combined with the approximately 2–3 cm precision floor of the rigid-transform action recovery, visible as the frequent reflection corrections (det(R) ¡ 0) during rollout, the policy can consistently navigate to the correct sub-region of the workspace but cannot reliably close the last few centimeters required to grasp objects whose graspable surfaces are themselves only a few centimeters wide as illustrated in Figure 5. This is exactly why the previously evaluated non-prehensile tasks (push cube, close drawer) achieved markedly higher success rates: their action requirements, translating the object or pushing a handle, are far more tolerant of these few-centimeter errors than a precision grasp. Our extended tasks are predominantly prehensile , so the same precision floor that was harmless before now compounds with the keypoint-identity drift to make contact with the graspable surface unreliable.

## B Model Architecture for Video-Conditioned $\pi_{0.5}$

Our video-augmented Pi-0.5 policy builds on the perceptual-memory and memory-as-context design introduced by Dai et al. [39], but repurposes the memory tokens for learning from a human demonstration video rather than from the robot's own past observations. Specifically, the policy is built on top of the public `openpi/pi05_base` checkpoint [57] and preserves its SigLIP-So400m/14

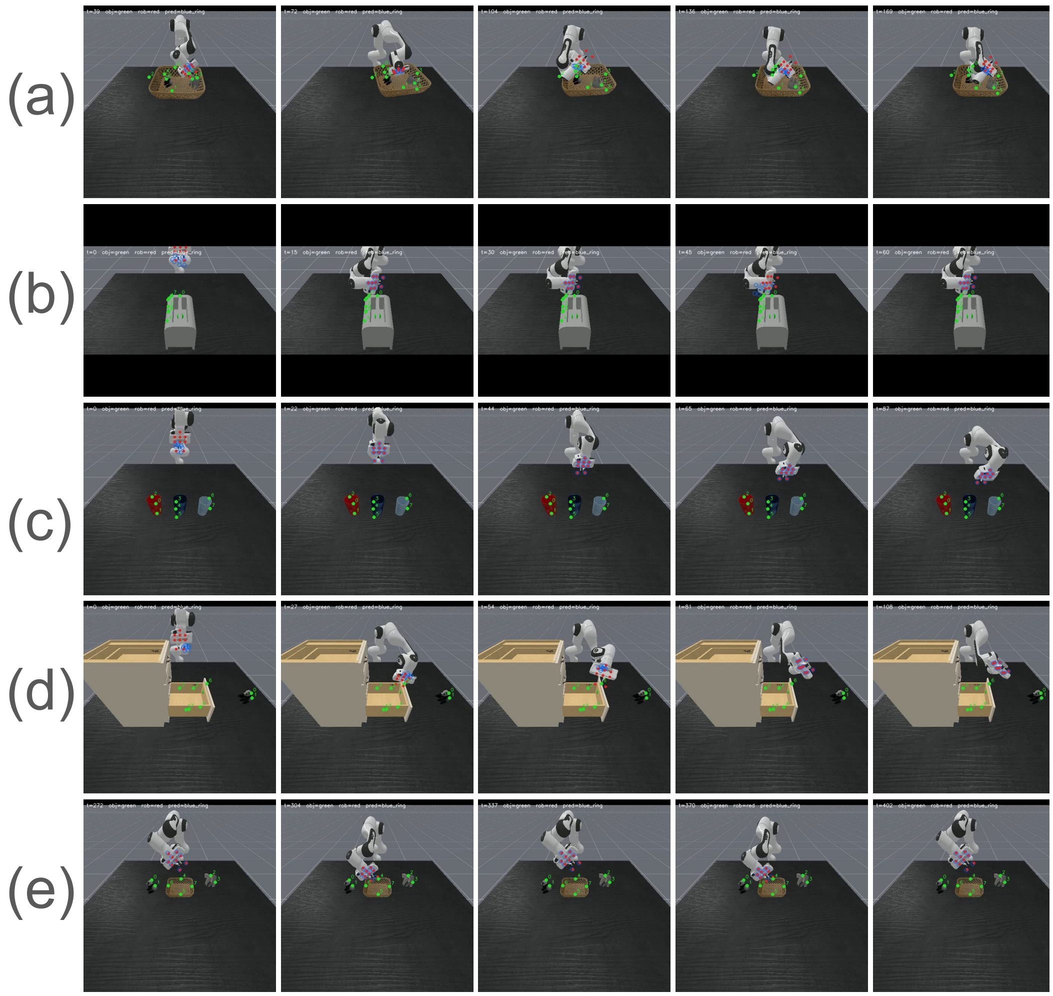

Figure 5: Qualitative rollouts for the extended tasks: (a) empty basket, (b) press a toaster button, (c) stack three cups, (d) place a toy in a drawer and close the drawer, and (e) place toys in a basket. In all tasks the policy follows the overall trajectory, moving toward the cups, toys, or button, but fails to execute the precise manipulation required to press the button or grasp the target object.

image tower, Gemma VLM expert, and flow-matching action expert. The only added component is a lightweight demonstration encoder that converts a human video into a fixed-length visual prefix. This prefix is prepended to the standard Pi-0.5 input sequence and is processed jointly with the robot observation and language tokens.

### B.1 Video-Conditioned Policy Formulation

We formulate demonstration-conditioned manipulation as action prediction from three sources of information: a human demonstration video, the robot's current multi-view observation history, and a generic language prompt. Let

$$D = \{I_1^h, I_2^h, \dots, I_{T_h}^h\}$$

denote a human demonstration video for the target task, where each $I_t^h$ is an RGB frame. Let

$$O_t = \{I_{t-k:t}^{r,v}\}_{v=1}^{V}$$

denote the robot observation history at control step $t$, consisting of multi-view RGB observations over an observation horizon of length $k{+}1$. The policy predicts an action chunk

$$A_t = (a_t, a_{t+1}, \dots, a_{t+H-1})$$

conditioned on the demonstration and the current robot observations:

$$\pi_\theta(A_t \mid D, O_t, \ell),$$

where $\ell$ is a fixed task prompt. In our experiments, we set $\ell =$"`complete the task`" for all tasks, so the human demonstration is the only task-specific conditioning signal.

The demonstration video is first mapped to a fixed-length sequence of demonstration tokens:

$$Z_D = g_\phi(D) \in \mathbb{R}^{B\times d},$$

where $B$=512 is the demonstration-token budget and $d$=2048 is the Pi-0.5 token dimension. The robot observations and language prompt are embedded by the original Pi-0.5 encoders into token sequences $Z_O$ and $Z_\ell$. The VLM expert receives the concatenated prefix

$$Z_{\text{prefix}} = [Z_D; Z_O; Z_\ell],$$

and the action expert predicts the velocity field used by the flow-matching objective:

$$v_\theta(x_t, t \mid Z_{\text{prefix}}),$$

where $x_t$ is the noisy action chunk at denoising time $t$. Training follows the original Pi-0.5 conditional flow-matching objective, with the demonstration tokens acting as additional visual context.

### B.2 Base Pi-0.5 Components

**Input tokenization.** The policy receives three types of tokens: demonstration tokens, robot observation tokens, and language tokens. Robot observations are encoded from the `base_camera` and `hand_camera` RGB streams. Language tokens encode the fixed prompt `‘‘complete the task’’`. All tokens are projected to the Pi-0.5 hidden dimension $d$=2048. Since the language prompt is identical across tasks, task identity must be inferred from the demonstration prefix.

**VLM and action experts.** We preserve the two-expert structure of Pi-0.5. The VLM expert fuses the visual and language prefix tokens, while the action expert predicts robot action chunks conditioned on the VLM features and the denoising timestep. We initialize both experts from `pi05_base` and fine-tune them jointly with the demonstration encoder.

**Attention structure.** The demonstration, robot-image, and language tokens form the prefix sequence. Attention is bidirectional within this prefix, allowing the model to jointly reason over the human video, the current robot scene, and the prompt. The action suffix contains the noisy action tokens and uses the standard prefix-LM structure: action tokens attend to the prefix and causally attend within the action sequence. We implement this token-level masking with the `ar_mask` and `na_mask` interface in the `LfOPi0` forward pass.

**Flow-matching action prediction.** The action expert predicts action chunks using conditional flow matching. A velocity field is trained to map Gaussian noise to the target action chunk conditioned on the prefix representation. The denoising timestep is sampled per batch element as $t \sim \text{Beta}(1.5, 1)$ and scaled to the interval $(0.001, 1)$, matching the Pi-0.5 training setup.

### B.3 Human Demonstration Encoder

The human demonstration encoder converts a single front-view human video into a fixed-length visual prefix. The prefix has a budget of $B$=512 tokens and is constructed from selected frames of the demonstration video.

**Offline visual feature extraction.** For efficiency, demonstration features are precomputed and cached before policy training. Each $224\times224$ front-view demonstration frame is passed through the SigLIP-So400m/14 image tower from the `pi05_base` checkpoint. The resulting patch features are average-pooled to a $4\times4$ spatial grid, yielding $P$=16 visual tokens per frame. Each token has dimension 2048. We also cache $8\times8$ and $2\times2$ pooled grids, but the deployed configuration uses only the $4\times4$ grid. The SigLIP image tower is frozen during this precomputation step.

**Spatiotemporal positional encoding.** Each demonstration token is augmented with a 3-D sinusoidal positional embedding that encodes its spatial and temporal location. The embedding has dimension 768 and is divided across the $y$, $x$, and $t$ axes using sinusoidal sine-cosine features. Spatial coordinates are computed with respect to the original $16\times16$ patch grid and then mapped to the pooled $4\times4$ grid so that each pooled token is positioned at the center of its corresponding patch group. Temporal coordinates are indexed by the absolute frame number in the demonstration video. Spatial frequencies use base 1,000, and temporal frequencies use base 10,000.

**Uniform frame sampling.** In the uniform variant, we sample $T_{\text{frame}}$=32 frames evenly across the demonstration video. Since each frame contributes $P$=16 visual tokens and we use a single human-video view, the resulting prefix length is

$$T_{\text{frame}} \cdot P \cdot V = 32 \cdot 16 \cdot 1 = 512,$$

which exactly matches the demonstration-token budget.

**BPP keyframe sampling.** We also evaluate a keyframe-aware sampling variant, BPP [59]. Given a set of task-relevant keyframes, all annotated keyframes are included whenever they fit within the frame budget. The remaining frame slots are sampled from the non-keyframe frames with probability inversely related to their distance from the nearest keyframe. Let $\mathcal{K}$ denote the set of selected keyframes and let

$$d_{\text{kf}}(i) = \min_{j\in\mathcal{K}} |i - j|$$

be the temporal distance from frame $i$ to its closest keyframe. Non-keyframe frames are sampled with weights proportional to

$$w_i \propto \frac{1}{d_{\text{kf}}(i) + \epsilon},$$

where $\epsilon$ avoids division by zero. Thus, frames closer to annotated keyframes are more likely to be selected. The uniform and BPP variants differ only in the frame sampling rule; all other architectural components are shared.

**Token construction.** For each selected demonstration frame, the pooled SigLIP feature and the 3-D positional embedding are combined into a Pi-0.5-compatible token. Let $f_{i,p} \in \mathbb{R}^{2048}$ be the SigLIP feature for patch $p$ in selected frame $i$, and let $e_{i,p} \in \mathbb{R}^{768}$ be its spatiotemporal positional embedding. The encoder computes

$$z_{i,p} = W_{\text{joint}} \left[f_{i,p}; \text{silu}(W_{\text{pos}} e_{i,p})\right],$$

where $W_{\text{pos}} : \mathbb{R}^{768} \to \mathbb{R}^{768}$ and $W_{\text{joint}} : \mathbb{R}^{2816} \to \mathbb{R}^{2048}$ are learned linear projections. State-embedding fusion is disabled in our configuration. The resulting sequence of tokens $\{z_{i,p}\}_{i,p}$ forms the demonstration prefix $Z_D$.

The demonstration encoder consists only of these two linear layers and adds approximately 6.5M parameters. It does not introduce additional transformer blocks, attention heads, or expert branches.

### B.4 Demonstration-as-Context Integration

The demonstration prefix is integrated by prepending it to the standard Pi-0.5 prefix sequence. Concretely, the VLM expert receives

$$[Z_D; Z_O; Z_\ell],$$

where $Z_D$ contains the human demonstration tokens, $Z_O$ contains the current robot observation tokens, and $Z_\ell$ contains the fixed language prompt tokens. These tokens are processed together by the PaliGemma stack under the prefix attention mask. The action expert then predicts the flow-matching velocity field conditioned on the resulting prefix features.

This integration mechanism treats the human demonstration as visual context for the current control problem. The model can therefore compare the human video with the current robot observation before predicting the next action chunk. Importantly, the action head, action horizon, denoising procedure, and flow-matching objective are unchanged from Pi-0.5.

**Summary.** Video-Augmented Pi-0.5 is Pi-0.5 with a length-512 visual demonstration prefix. The prefix encodes a 32-frame human demonstration video using frozen SigLIP features, spatiotemporal positional embeddings, and a small linear projection module. Apart from this demonstration encoder and the prepended visual prefix, the underlying Pi-0.5 architecture is left unchanged.

# C Training Details

Each model is trained twice with identical hyperparameters: once on the no-distraction training set and once on the with-distraction training set (see the main paper). Unless otherwise specified, all runs use a single NVIDIA H200 GPU with 144 GB memory and seed 42. We save checkpoints every 10k steps and report results from the final checkpoint. The final checkpoint is at 50k steps for all variants except the Lerobot Pi-0.5 FFT language-conditioned baseline, which is trained for 30k steps.

## C.1 Vid2Robot

We implement Vid2Robot following the description in the original paper [16]. The visual backbone is a pretrained ViT-B/16 at $224 \times 224$ resolution. The prompt and state streams each pass through a 2-layer Perceiver Resampler with 64 latents, 12 heads, and head dimension 64. The State-Prompt Encoder and Action Decoder are 4-layer Transformers with hidden dimension 768 and 8 attention heads. The model receives $T_{\text{prompt}}$=16 prompt frames and $T_{\text{state}}$=8 state frames. The action head predicts a chunk of $C$=4 actions, with each action dimension discretized into 256 bins for the 8-D `pd_joint_pos` action space.

The training objective is the mean of four losses: (i) cross-entropy over action tokens, (ii) Temporal Cycle Consistency between prompt and robot frame projections, (iii) prompt–robot video-video SigLIP loss, and (iv) video-text SigLIP loss against a SigLIP text-tower embedding of the task string. Optimization uses AdamW with peak learning rate $3 \times 10^{-4}$, final learning rate $1 \times 10^{-6}$, weight decay 0.05, linear warmup for 2,000 steps, and cosine decay to the final learning rate. We train with batch size 32 for 50,000 steps. To fit training on a single H200, we apply per-block gradient checkpointing to the ViT.

## C.2 UniSkill

We use the public UniSkill Inverse-Skill-Dynamics (ISD) encoder off the shelf, without fine-tuning on our data. The encoder weights are loaded from the authors' released pretrained checkpoint. This follows the protocol of the original paper, where the pretrained ISD checkpoint is not fine-tuned on the target human data. We run the encoder once over every robot frame and every human demonstration in the LfO data to precompute a 64-d skill-goal embedding for each clip. Only the downstream policy is trained.

**Downstream diffusion policy.** We train a Robomimic diffusion policy conditioned on the precomputed skill-goal embedding. The visual encoder consists of two ResNet-18 SpatialSoftmax heads, each with 32 keypoints, applied to the front-view and wrist-camera observations. We use random $116 \times 116$ crops. Low-dimensional inputs include gripper and joint states. The denoiser is a conditional 1-D UNet with channels $[256, 512, 1024]$, kernel size 5, 8 groups, and diffusion-step embedding dimension 256. The model is trained with DDPM-style noise prediction using 100 training timesteps and evaluated with DDIM using 10 inference steps. We use a squared-cosine-cap-$v_2$ noise schedule with `clip_sample=True`. The observation, action, and prediction horizons are 2, 8, and 16, respectively.

EMA is enabled with power 0.75. Optimization uses Adam with learning rate $1 \times 10^{-4}$, no $L_2$ regularization, batch size 256, and 1,000 epochs of 100 iterations each, for a total of 100,000 gradient steps. We use seed 0. Actions are normalized per dimension using min-max statistics. We use the `skill_aug` setting with `aug_num=5`, where the skill embedding is replaced by a same-task neighbor with probability determined by the augmentation count.

**Example Prompt for Annotating Keyframes with Big-Picture Policy(BPP)**

**System Message**

**Text Prompt**

You are a robot manipulation analysis assistant. Identify keyframes — frames that capture a distinct, task-relevant state transition: gripper opens or closes, contact with an object is established or released, an object starts or stops moving, or a sub-goal is reached. Return ONLY a JSON object with a single key k̈eyframesẅhose value is a list of integer frame indices from those shown. Do not include any other text.
'Example: "keyframes": [0, 42, 111]'

**User Message**

**Text Prompt**

Task: `<Task Description>`. Below are `<Number of Frames>` sampled from the episode. Each frame is labeled with its index in the original video.
`<List of Frame Indices>`

**Text Prompt**

Identify the keyframes for this task. Return the JSON object only.

**Input Images**

`<A list of frames from a human demonstration video>`

---

**Assistant Response**

{"keyframes": [0, 30, 247]}

Figure 6: Example prompt for generating keyframe annotations using the VLM-based keyframe annotation framework. This framework is inspired by BPP, and we use GPT-5.4-mini to as the backbone VLM. The downstream video-conditioned $\pi_{0.5}$ VLA policy select frames from the videos using these keyframe annotations.

### C.3 Video-Conditioned $\pi_{0.5}$ (Uniform Frame Sampling)

The video-conditioned Pi-0.5 variant fine-tunes the public Pi-0.5 base checkpoint with our demonstration adapter. We represent the human demonstration using a token budget of 512 and a frame budget of 32, with 16 tokens per image. The adapter uses an observation horizon of 16, mean pooling, a 2048-d demonstration-token dimension, enabled positional embeddings, and disabled state embeddings. Demonstration frames are sampled uniformly across the human video. Robot observations use both the front-view and wrist cameras, while human demonstrations are collected from the front-view camera.

Optimization uses JAX FSDP on one device with AdamW ($\beta_1$=0.9, $\beta_2$=0.95, $\epsilon$=$10^{-8}$, weight decay $10^{-10}$), gradient-norm clipping at 1.0, and a cosine learning-rate schedule with peak learning rate $5\times10^{-5}$. The schedule uses 1,000 warmup steps and decays to $0.1\times$ the peak learning rate. EMA is applied to the weights with decay 0.999. We train with batch size 64 for 50,000 steps using seed 42. The text prompt is fixed to "*complete the task*", so the model receives no language information beyond the demonstration video.

### C.4 Video-Conditioned $\pi_{0.5}$ (BPP keyframe sampling)

The BPP-$\pi_{0.5}$ variant uses the same architecture and optimization settings as the uniform frame-sampling variant. The only difference is the human-demonstration frame sampling strategy. For this variant, we use a keyframe annotation framework inspired by BPP to precompute keyframe

**Example Prompt for Using VLM to Annotate Human Videos**

**User Message**

**Input Images**

<A list of frames from a human demonstration video>

**Text Prompt**

These images are frames sampled in chronological order from a video of a person performing a manipulation task. Write the complete and clean task instruction for this video so a different human operator can reproduce the task without looking at the video.

**Assistant Response**

Place the panda-shaped object into the bottom drawer of the white mesh organizer.

Figure 7: Example prompt for using VLM to generate language annotation for human demonstration videos. The generated language instructions are consumed by the downstream $\pi_{0.5}$ VLA policy.

annotations and assign each frame a sampling weight based on its distance to the closest selected keyframe. Frames closer to selected keyframes receive higher sampling weights.

Fig. 6 describes the prompt we use to annotate keyframes with a self-implemented VLM framework inspired by BPP framework [59]. We use GPT-5.4-mini [61] as the backbone VLM to select keyframes for input human demonstration videos. The original BPP framework use all frames of the videos as inputs for keyframe annotations. To reduce the computational cost, we downsample the human videos to 2-fps uniformly, and provide the list of indices for the selected frames for the VLM. The VLM then returns the selected keyframes from the list of candidate frames in the json format.

### C.5 Language-conditioned $\pi_{0.5}$(VLM Upstream)

This baseline is the language-conditioned $\pi_{0.5}$ variant, and does not take in video inputs. We fully fine-tune language-conditioned $\pi_{0.5}$ using the lerobot implementation. Training uses robot trajectories with base and wrist cameras at $224\times224$ resolution, robot state, and action chunks. We finetune the action expert of the $\pi_{0.5}$ model and keep the VLM expert frozen during training. The language instruction for each robot sample is generated by Qwen3-VL-4B from the corresponding human demonstration of the same task. Fig. 7 shows the prompt we use to invoke the VLM, and an example response from the VLM model. We use the huggingface checkpoint for video annotations.

Optimization uses PyTorch AdamW with peak learning rate $2.5\times10^{-5}$ and cosine decay to $0.1\times$ the peak learning rate using cosine annealing LR over the full training schedule, without a separate warmup phase. The optimizer $\beta$ values, $\epsilon$, and weight decay are inherited from the lerobot preset. Gradients are clipped using the threshold specified by the configuration. We train with batch size 32, bfloat16 precision, gradient checkpointing, 30,000 gradient steps. Input normalization statistics are estimated from 5,000 randomly sampled training batches before training begins. We store checkpoints at every $10,000$ steps and evaluate the last checkpoint.

### C.6 PointPolicy

We follow Point-Policy [23] to train an imitation policy directly from human demonstrations. We extract keypoint trajectories from the human videos and retarget them into the robot end-effector trajectories in the calibrated environment. These end-effector keypoints, together with the task-

| Dataset Name | Total #Tasks | Real-world Human Demo. #Hours | Total #Human Demo. | Avg. #Secs Human Demo. | Total #Robot Demo. | Avg. #Steps Robot Demo. |
|---|---|---|---|---|---|---|
| RoboCerebra [42] | 1,060 | N/A | N/A | N/A | 1,000 | 2,972 |
| LIBERO [40] | 130 | N/A | N/A | N/A | 6,500 | 162 |
| RoboCasa [46] | 100 | N/A | N/A | N/A | 100K+ | - |
| VLABench [49] | 100 | N/A | N/A | N/A | 1,600 | 115 |
| MimicGen [43] | 12 | N/A | N/A | N/A | 48K+ | - |
| EgoVerse [50] | 1,965 | 1,362 | 79,620 | - | N/A | N/A |
| Human2Robot [9] | 8 | - | 2,600 | - | 2,600 | - |
| RH20T [27] | 147 | - | 110K+ | - | 110K+ | - |
| MimicDroid [26] | 12 | N/A | N/A | N/A | N/A | - |
| RoboReel | 10 | 9 | 3,000 | 11.56 | 2,000 | 304 |

Table 5: **Overall statistics comparison between RoboReel and other existing benchmarks.** Compared to existing datasets and benchmarks, RoboReel features paired real-world human demonstration videos and portable and scalable evaluation environment. This allows a fair comparison for learning-from-observation frameworks using our benchmark.

| | Human videos | | | | | | Robot trajectories | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | Counts | | | | Duration (s) | | Counts | | | Length (steps) | |
| Task | Total | ND | WD | PD | Mean | Std | Total | ND | WD | Mean | Std |
| bowl_in_plate | 300 | 100 | 100 | 100 | 12.68 | 3.97 | 200 | 100 | 100 | 262 | 19 |
| close_drawer | 300 | 100 | 100 | 100 | 7.46 | 1.55 | 200 | 100 | 100 | 185 | 22 |
| empty_basket | 300 | 100 | 100 | 100 | 14.01 | 4.43 | 200 | 100 | 100 | 498 | 57 |
| open_lid | 300 | 100 | 100 | 100 | 10.46 | 1.24 | 200 | 100 | 100 | 268 | 5 |
| pick_cube | 300 | 100 | 100 | 100 | 7.09 | 2.41 | 200 | 100 | 100 | 139 | 14 |
| press_toaster | 300 | 100 | 100 | 100 | 5.72 | 1.61 | 200 | 100 | 100 | 112 | 21 |
| push_cube | 300 | 100 | 100 | 100 | 5.73 | 0.94 | 200 | 100 | 100 | 135 | 6 |
| stack_cups | 300 | 100 | 100 | 100 | 17.84 | 1.79 | 200 | 100 | 100 | 515 | 50 |
| toys_in_basket | 300 | 100 | 100 | 100 | 17.44 | 3.90 | 200 | 100 | 100 | 436 | 36 |
| toys_in_drawer | 300 | 100 | 100 | 100 | 13.04 | 2.61 | 200 | 100 | 100 | 489 | 29 |
| **Total / Avg.** | **3000** | **1000** | **1000** | **1000** | 11.56 | — | **2000** | **1000** | **1000** | 304 | — |

Table 6: Per-task dataset statistics for human video demonstrations and robot trajectories. For each task, human videos include a total of 300 demonstrations. Each of the ND, WD, and PD condition has 100 demonstration videos. Robot trajectories include 200 demonstrations, including 100 trajectories for the ND and WD conditions each respectively. Human-video duration is reported in wall-clock seconds from the `front` camera at 30 fps, and robot-trajectory length is reported as the number of discrete simulator control steps at 30 Hz. The final row reports total counts and the average per-task duration or trajectory length.

specific object keypoints, are used to train the policy. Given the current end-effector keypoints and object keypoints, the model predicts the next end-effector keypoints. The details of the keypoint annotation and the mapping to simulation of the robot actions are described in Appendix D.2.

The PointPolicy baseline is trained independently for each task. We train the policy for 90,000 gradient steps with a batch size of 128, using AdamW optimizer. Predictions are executed using a waypoint-based trajectory controller. Each prediction defines the next target waypoint, and the controller drives the end-effector toward it until the waypoint is reached or a per-waypoint timeout elapses. The policy is then queried again to generate the next waypoint. This process continues until the task is completed or the maximum episode length is reached. The PointPolicy is evaluated in the simulated environments under two settings, with distraction objects and without distraction objects. We use the same set of distraction objects for the PD environment as the WD and ED environments. The details are described in Appendix D.

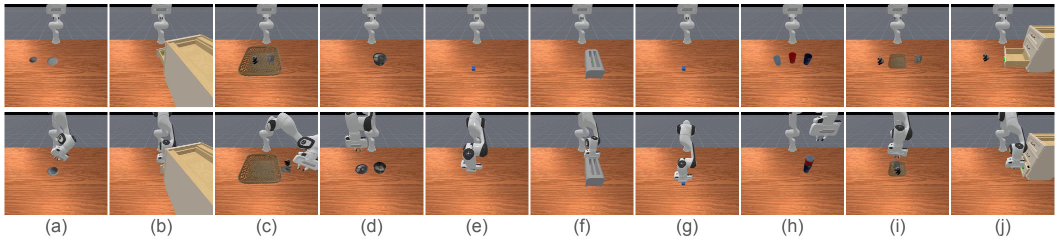

Figure 8: Examples of start and end configurations of a successful instance in the non-calibrated evaluation environment for each task– (a) place a bowl in a plate, (b) close drawer, (c) empty basket, (d) open a lid, (e) pick a cube,(f) press a toaster button, (g)push cube, (h) stack three cups, (i) place toys in a basket, (j) place a toy in a drawer and close the drawer. The top half of this figure shows the initial states and the bottom half shows the end states.

# D Details of the Benchmark

Table 5 shows the comparison of the statistics between RoboReel and existing datasets and benchmarks for robot learning and learning from human videos. Compared to previous works, RoboReel provides paired real-world human videos and simulation trajectories for training, and paired task environments for fair and reproducible comparison for models from various model families on manipulation tasks of different categories.

## D.1 Dataset Statistics

We report descriptive statistics for the benchmark in Table 6. This table provides detailed statistics for both human demonstration videos and robot trajectories. The statistics of the human videos are reported in seconds and the statistics of the robot trajectories are reported in the number of discrete time steps.

**Human demonstration videos.** Each task–condition pair includes 100 human demonstrations of the same task performed by a human operator in front of a fixed table-top rig, recorded simultaneously from *four* cameras (`egocentric`, `front`, `left`, `right`) at $1280 \times 720$ resolution and 30 fps. Across the 10 single-step tasks the dataset totals 3,000 demonstrations (12,000 mp4 files) spanning roughly 9 hours of clock time at the `front`-camera view (the view our models consume).

**Robot trajectories.** Each task–condition pair includes 100 robot trajectories collected in ManiSkill 3 / SAPIEN with the same scene configuration and object identity as the corresponding human demonstrations. Every trajectory is saved as one HDF5 holding the per-step robot state (`obs`, 36-d), 8-d absolute `pd_joint_pos` actions, and the simulator's full environment state, plus five synchronized $1280 \times 720$ RGB MP4s (`front_camera`, `hand_camera`, `left_camera`, `overhead_camera`, `right_camera`) at 30 fps. Across the 10 single-step tasks the dataset totals 2,000 trajectories.

## D.2 Additional Descriptions on the Benchmark

Table 7 provides a natural language description for each task, along with the details of the implementation of the success criteria in the simulated environment. Fig. 8 shows the start and end images in the non-calibrated simulation environment. The top half of the figure shows the images of the initial states and the bottom half shows the last states of a successful execution. Fig. 9 shows frames from the original human videos, human videos with keypoint annotations, the non-calibrated task environments, and the calibrated task environments from top to bottom, respectively. Fig. 11 shows the images of the objects we use to collect real-world human videos. The first part of the figure shows the image of the task-relevant objects and the second part shows the distraction objects.

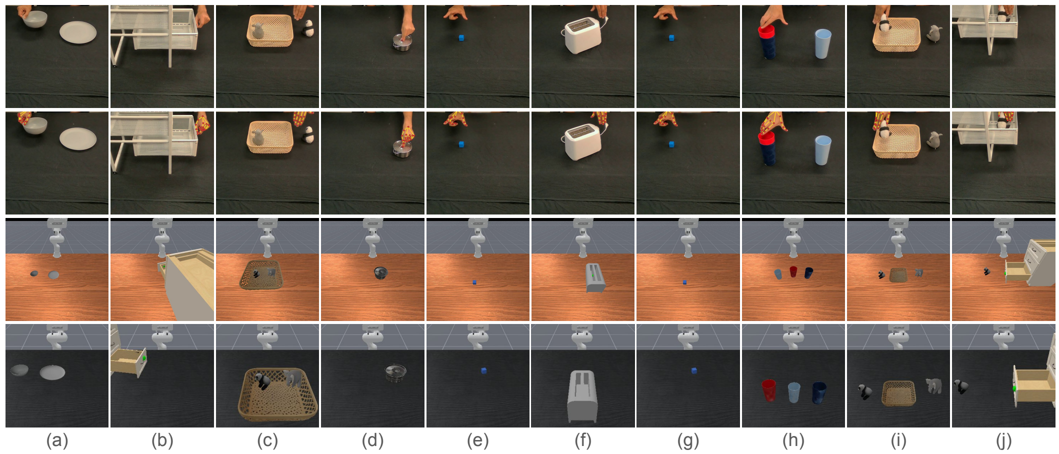


Figure 9: From top to bottom are frames from original human videos, human videos with keypoint annotations, non-calibrated task environment, and calibrated evaluation environments for each task – (a) place a bowl in a plate, (b) close drawer, (c) empty basket, (d) open a lid, (e) pick a cube,(f) press a toaster button, (g)push cube, (h) stack three cups, (i) place toys in a basket, (j) place a toy in a drawer and close the drawer.

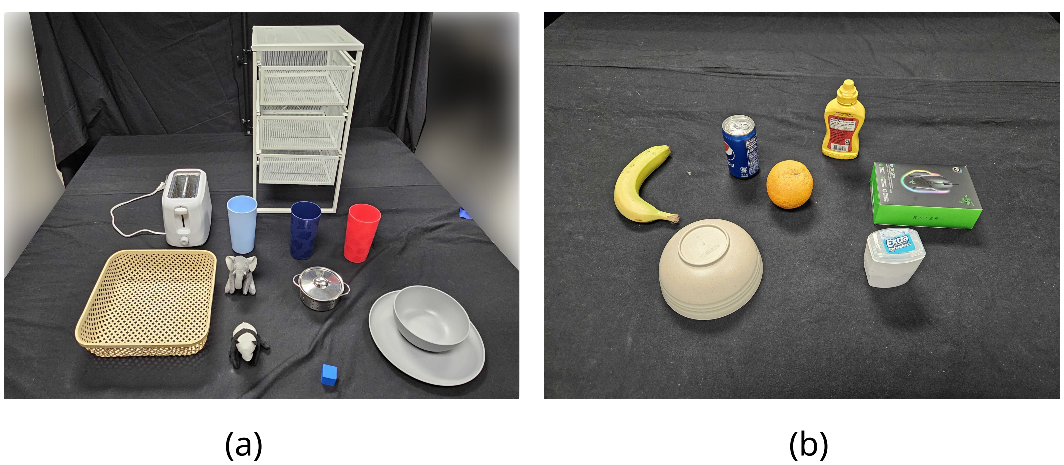


Figure 10: This figure shows the real-world objects we use. (a) The image of all the task relevant objects, including a toaster, a basket, a white drawer, three cups of different colors, an elephant soft toy, a pot and its lid, a panda soft toy, a blue cube, a gray bowl, and a gray plate. (b) The image of the real-world distraction objects, including a banana, a can of coke, a flipped bowl, an orange, yellow mustard, a mouse box, and a chewing gum box.

| Task | Description | Success Criterion |
|---|---|---|
| `bowl_in_plate` | Pick up the bowl and place it on the plate. | The final position of the bowl is located on top of the plate. The xy-coordinate of the bowl is within the area of the plate, and the z-coordinate of the bowl must be on top of the plate. |
| `close_drawer` | Close the bottom drawer. | The drawer has been pushed inward by at least half of its open range. |
| `empty_basket`$^{\dagger}$ | Remove the two toys from the basket in any order. | Both toys are located outside the basket. Both toys must have final locations where either the xy-coordinates are outside the basket area, or the z-coordinates are beyond the height of the basket. |
| `open_lid` | Pick up the lid on the pot and place it on the table. | The lid is on the table and not on top of the pot. The xy-coordinate of the lid must be outside the pot area, and the z-coordinate of the lid must be between the table and the pot. |
| `pick_cube` | Lift the cube above the table. | The z-coordinate of the cube is at least 3cm above the table. |
| `press_toaster`$^{*}$ | Press the toaster down to the bottom. | The toaster's slider joint travels downward at least 80% of its total range. |
| `push_cube` | Push the cube away from its original position. | The cube has traveled for at least 5cm initial position at any direction on the xy-plane. |
| `stack_cups`$^{*\dagger}$ | Stack the red cup on the dark blue cup, then stack the light blue cup on top. | The three cups are arranged in the required vertical order: dark blue at the bottom, red in the middle, light blue on top. All cups have the same xy-coordinate, and z-coordinates with a small distance following the order. |
| `toys_in_basket`$^{\dagger}$ | Pick up the two toys and place them in the basket in any order. | Both toys are located inside the basket. Both toys must have final locations where the xy-coordinates are within the basket area, and the z-coordinates are below the height of the basket. |
| `toys_in_drawer`$^{\dagger}$ | Pick up the panda toy and place it in the drawer, then close the drawer. | The panda toy is located inside the open drawer, and the drawer is closed for at least half of its open range. The panda toy has a final position where the xy-coordinate is within the drawer area, and the z-coordinate is between the height of the lowest two drawers. The drawer has been pushed inward by at least half of its open range. |

Table 7: **Detailed descriptions for each task.** We include the natural language task description for each task and the success criterion in the simulation. The $\dagger$ superscript denotes long-horizon task, and the $*$ superscript denotes high-precision tasks.

**Keypoint Annotation for Human Videos.** Human demonstrations are processed through DIFT [62] to perform semantic object localization on the first frame of each demonstration, while CoTracker [63] propagates these keypoints to produce continuous keypoint trajectories throughout the demonstration. We apply a low-pass filter to the recovered hand poses to reduce articulation-induced noise and suppress 180° orientation flips that the rigid transformation can introduce on

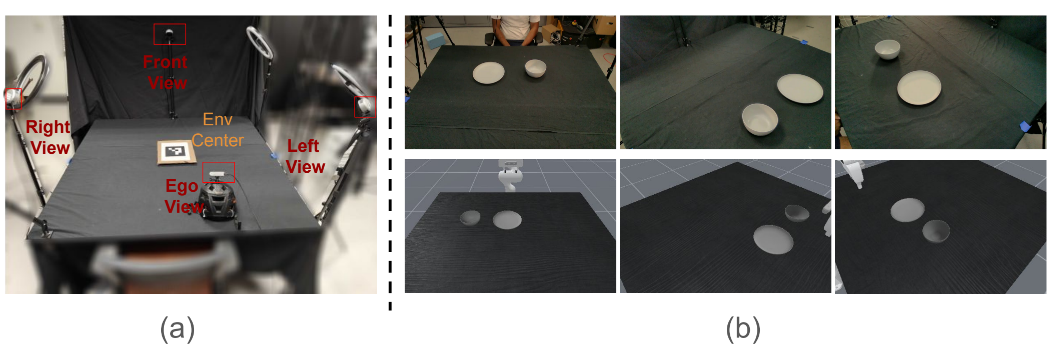


Figure 11: This figure shows the camera settings of the PD test suit($\diamondsuit$).(a) The four cameras in the real-world data collection setting. The ArUco marker marks the center of the table as the origin. (b) The corresponding camera views in the real world and simulation. From left to right we show the view from the front camera, the left camera, and the right camera. The top row shows the camera views from the real world, and the bottom row shows the camera views in the simulation. We compute the 3D camera poses with respect to the origin in the real world, and place the cameras at the corresponding poses in the simulation.

non-rigid hand landmarks. In the pose-defined setting where the action space of the simulation environment is calibrated to the real-world environment, these extracted 3D keypoint trajectories can be directly mapped to the end-effector keypoints of the robot in the simulation environments. We use these trajectories as the action annotations to train our pose-defined baseline.

### D.3 Creating Paired Task in Simulation.

To construct a faithful digital twin of every real-world task, we ported each tabletop setup into ManiSkill 3 / SAPIEN with a 7-DoF Franka Panda arm. Three design objectives guided the construction: (i) *visual fidelity*, so that policies conditioned on real human videos see matching colors, shapes, and scene layouts at evaluation time; (ii) *geometric and kinematic fidelity*, so that contact-rich behaviors transfer; and (iii) *procedural reproducibility*, so that anyone can regenerate the paired robot data with a single command. The following paragraphs describe each component of this pipeline.

**Asset reconstruction and import.** Rigid-body assets that appear in the real-world rig (cups, bowls, plates, lids, toys, the toaster cube, the cube, and the bin) are reconstructed as digital twins with SAM3D-Objects directly from RGB captures of the physical objects. Each reconstructed asset is exported as a triangle mesh for rendering and a V-HACD convex decomposition for collision, allowing accurate physics while preserving fine visual detail (color, texture, decals). Articulated assets, namely the drawer used in `CloseDrawer` and `ObjectInDrawer` and the toaster used in `Toaster`, are pulled directly from PartNet-Mobility, selecting in each case the asset that most closely resembles the corresponding real-world object in appearance and joint structure, so that the joint axes, joint limits, and link hierarchies are physically grounded. For every asset we precompute an upright resting orientation, an axis-aligned half-size, and a footprint radius (stored as `orientation_result.json`), which the spawner uses to seed each episode with stable, collision-free initial poses.

**Scene and camera calibration.** The simulated table inherits the dimensions of the physical table. Cameras are placed using the same ArUco-based extrinsics solved on the physical rig: a `front/front` camera at $(0.4, 0, 0.6)$, side cameras at $(0.4, \pm 0.4, 0.6)$, and an `overhead` camera at $(0, 0, 0.6)$, all looking at the table center. Each sensor renders $1280 \times 720$ RGB at 30 fps with the respective FoV, mirroring the real-world stream that the LfO models consume.

**Procedural trajectory generation.** Robot trajectories are produced by task-specific solvers built on top of `mplib`'s RRT-Connect and screw-motion planners (wrapped in `FixedPandaMotionPlanningSolver`). For *pick-and-place* tasks (`PickCube`, `PutBasket`, `BowlOnPlate`, `StackCups`, `ObjectInDrawer`), GraspGen runs on per-object point clouds extracted from the current scene to propose candidate grasps; candidates are then re-ranked by a weighted cost combining angular distance from the current TCP, deviation from a downward approach, and joint-space displacement. The chosen grasp is executed through a five-phase template, pre-grasp, grasp, transport, pre-place, drop, with randomized standoffs along the approach axis to inject grasp diversity. For *articulated* tasks (`CloseDrawer`, `Toaster`), the solver reads the live joint axis from the PartNet-Mobility model and waypoints the end-effector along that axis with a fixed standoff. Each trajectory is verified against the env's `evaluate()` success predicate before recording.

**Distractor and texture randomization.** `DistractorMixin` class is composed into every environment and, when `distract=True` is called, it integrates 1-2 kinematic distractor assets per episode from an asset library of 5 task-irrelevant assets (banana, bowl, mouse box, orange, soda can), placing them by rejection sampling with a 25 cm clearance from any task-relevant object. To support deterministic replay of evaluation banks, the spawner also honors an explicit `distractor_names` override so that recorded `state_dicts` reproduce the exact same asset identities.

**Data format and reproducibility.** Each successful trajectory is logged via `RecordEpisode` as a single HDF5 holding 36-dimensional proprioceptive observations, 8-dimensional absolute `pd_joint_pos` actions, and the full simulator state, together with five synchronized $512\times512$ MP4 streams (left, right, overhead, front and wrist). Table 8 summarizes the solver strategy and primary asset source per task.

| Task | Solver strategy | Primary asset source | Articulated |
|---|---|---|---|
| PushCube | Scripted contact + screw motion | Procedural primitive | ✗ |
| PickCube | GraspGen + 5-phase plan | Procedural primitive | ✗ |
| BowlOnPlate | GraspGen + 5-phase plan | SAM3D-Objects | ✗ |
| StackCups | GraspGen + sequential 5-phase plan | SAM3D-Objects | ✗ |
| PutBasket | GraspGen + sequential drop into bin | SAM3D-Objects | ✗ |
| EmptyBasket | GraspGen + sequential lift-and-place | SAM3D-Objects | ✗ |
| ObjectInDrawer | GraspGen + screw motion along joint axis | SAM3D-Objects + PartNet-Mobility | ✓ |
| CloseDrawer | Waypoints along live prismatic joint axis | PartNet-Mobility | ✓ |
| Toaster | Waypoints along live prismatic joint axis | PartNet-Mobility | ✓ |
| OpenLid | Contact-based screw motion | SAM3D-Objects | ✗ |

Table 8: Per-task summary of the paired simulation pipeline. Every task is implemented with the same Franka Panda arm, table geometry, and camera calibration as the real-world rig; only the solver template and asset source vary.

### D.4 Evaluation Protocol

We train one model checkpoint for each model class under each configuration. Our experiment results are reported from three evaluation runs, and each run uses a different set of random seeds to create the evaluation environments. The success rate for each individual evaluation run is computed from 20 rollouts with different seeds. We report the means and standard error of means (SEM) of the three evaluation runs for each model in each table. More specifically, the number of each grid of each table is computed from

$$3\ \texttt{evaluation runs}\ \times 20\ \texttt{rollouts per evaluation runs} = 60\ \texttt{total rollouts}.$$

**Pre-recorded evaluation banks.** To guarantee reproducibility, we collect 100 environment seeds for each task under each configuration, and we use the first 60 seeds to spawn the evaluation environments. For every `(task, env_distract)` pair we pre-record an *evaluation bank* of 300 entries and ship it with the benchmark. Each entry stores (i) the ManiSkill RNG seed used to initialise the episode, (ii) a full `state_dict` snapshot capturing all object poses, robot joint positions, and velocities at that initial configuration, and (iii) for the with-distraction banks, the identities of the sampled distractor objects so they can be reproduced exactly. Entries are collected by an *oracle-filtered* procedure: the environment is reset with a fresh seed, the physics are allowed to settle, and the state is snapshotted; a task-specific oracle solver then attempts the task from that state. Only episodes in which the oracle *succeeds* are admitted to the bank — failed configurations are discarded and a new seed is tried. This guarantees that every bank entry is a genuinely solvable starting configuration. At evaluation, the protocol driver *replays* bank entries rather than re-seeding the env, which means:

1. Two different models see numerically identical starting states on every rollout, removing any variance from RNG-driven object spawns or distractor sampling.
2. For the with-distraction cells, the bank also pins the distractor *identities* via the `distractor_names` override (Sec. D.3), so the same set of clutter objects appears regardless of which model is being evaluated.
3. Banks are larger than any single pass needs, which allows the three-pass procedure described below to slice them into disjoint sub-banks.

**Evaluation Details for ND(♠), WD(♡), and ED(♣).** The models for these three test suites use human videos as inputs during evaluation. For each configuration, we use 80 of the 100 human demonstration videos for training, and the remaining 20 videos for evaluation. These 20 evaluation human videos are the same across the three evaluation runs, but they are paired with environments spawned with different seeds for different evaluation runs.

**Evaluation Details for PD($\diamondsuit$).** The model from the PD test suit does not take human videos as inputs for inference. We use all the 100 human videos with keypoint annotations for training. During evaluation, the model only receives inputs from the evaluation environments.

## E Extended Related Work

**Human Action Video Datasets.** The computer vision community has long been interested in understanding human action videos that can even have paired language commands with semantic annotations for actions taken by humans [28, 29, 30, 31, 32]. These datasets have paired language commands with semantic annotations for actions taken by humans as well [32, 28, 64, 30, 29]. Prior work [32, 28, 64] frames action understanding from human activity videos as a classification problem, assigning a single action label to each video clip. ActivityNet [30] introduce semantic taxonomy and include temporally localized activity labels for untrimmed videos, allowing models to learn action labels of different levels of granularity, and the temporal relationship between action segments. As the vision models scale up, other annotations such as object movements [29], personal bounding boxes and atomic action labels [65], and language instructions [31] are introduced to enhance models' ability to understand the spatio-temporal relationship of actions. Egocentric datasets such as Assembly101 [33], Epickitchen[35] or Ego-Exo4D [34] capture long-horizon, unscripted daily activities from the first-person viewpoints. Unfortunately, none of this data allows accurate comparisons for robotics community due to the lack of a paired environment for the robot to learn actions or skills. Ego-Exo4D [34] provides synchronized first-person and third-person videos for human activities, along with fine-grained action annotations, including language, pose, gaze, audio, keysteps, and expertise level. This dataset provides the foundation to train and evaluate foundation models on human action understanding. These human action dataset from the CV community are designed to train and evaluate vision and language models on reasoning over human activities, but they do not care for whether the actions in the videos can be learned and reproduced by an embodied agent in a different environment.

**Learning from Observation Methods.** Based on different knowledge transfer pathways, existing human-video-based techniques can roughly be organized into three different categories [6], including task-oriented transfer [14, 15, 10, 47], observation-oriented transfer [16, 18, 19, 20], and action-oriented transfer [26, 48, 22, 24, 23, 21, 17]. *Task-oriented transfer* extracts semantic task information from human demonstration videos, then uses the extracted task information to control robot policies to perform the same tasks. There are two major lines of work for task-oriented transfer, including explicit task structure transfer, and implicit task intent transfer. Explicit task structure transfer methods decompose a human demonstration video into temporal language instructions, and implicit task intent transfer methods learn to extract latent task intent representation by aligning human demonstration video with robot trajectories or natural language instructions by task intents [10, 12]. *Observation-oriented transfer* focuses on directly bridging the visual gap between the human demonstration videos and the perception of the robot embodied agents. This line of work either directly transforms human demonstration videos into embodiment-agnostic or robot-like visual formats in the pixel space [11, 66], or learn embodiment-invariant visual embeddings to align the observation space between human and robot [19, 16, 18]. *Action-oriented transfer* aims to distill the action information, such as hand affordances and object affordances, and uses the action information to either train or guide robot policies. Similar to the other transfer mechanism, action-oriented transfer can be divideed into explicit affordances transfer and implicit latent actions transfer. Explicit affordances transfer extracts affordances, such as 3D hand-positions [24, 22, 26] and object geometric [48], from the human demonstration videos, and uses these actions to directly control the robot [24, 48, 23] or to train robot policies [22, 26]. This line of work requires calibration of the observation spaces and the action spaces between the human demonstrator and the embodied agents to produce any meaningful downstream robot policy. On the other hand, implicit action transfer learns compact and embodiment-agnostic action priors directly from human videos by training forward dynamic models (FDMs) and inverse dynamic models (IDMs) using future observation reconstruction [21, 67, 68, 17]. As a result of the significant variance in assumptions, architectures, and evaluations of these methods, there is not a unified benchmark for systematic evaluation on the cross-embodiment learning from observation problem. We aim to address this gap by constructing this benchmark, and conducting experiments with selected model from each of these transfer mechanisms.